

김상옥 Kim Sang-Ok

SW QA Engineer · 4년 차

“표준이 있는 곳에선 정착시키고, 없는 곳에선 만든다.”

QA 프로세스를 0에서 1로 만들고, 반복은 자동화하고, 필요한 도구는 직접 만드는 SW QA 엔지니어입니다. 자동차 · 공공/교육을 거쳐 현재는 산업 AI 비전 검수 플랫폼 **Daisy**의 품질을 책임지고 있습니다.

☎ 010-4482-2033

✉ kx2471@gmail.com

💻 github.com/kx2471



전체 구성

다섯 개 챕터로 구성되어 있습니다. 아래 카드를 누르면 해당 챕터로 바로 이동합니다. 상단 챕터 버튼과 M 키 목차로도 언제든지 이동할 수 있습니다.

01 QA 프로세스 & 워크플로우 설계

Git x 이슈 트래커 x Automation 통합 설계, 배포 게이트(NO-GO 차단권), RC 검증 절차, 근본원인 분류 체계 수립.

프로세스 설계

Release Management

RCA

02 테스트 자동화

MLOps 파이프라인 전 구간 E2E 회귀 시스템 설계, 팀 셀프서비스 러너, Playwright/Selenium 도입 사례.

Playwright

병렬 설계

QA 이네이블먼트

03 AI 활용 & 도구 개발

Claude Agent 기반 Regression TC Creator — RCA 데이터를 회귀 TC로 바꾸는 워크플로우 자동화.

Claude Agent

MCP

LLM 워크플로우

04 경력 & 프로젝트

3개사 경력 타임라인과 프로젝트 10건 — Daisy 플랫폼과 고객사 도입 검증, 자율주행·커넥티드카 국책과제, 공공 웹, 자동차 내비게이션.

ASPICE

KOLAS

실차 검증

05 실무 · 산출물 · 도구

SWR/TC/Defect/Xray 등 QA 표준 산출물, API·보안 검증, 직접 만든 업무 자동화 도구, 기술 스택.

Xray

Confluence

Python 도구

마지막에 토이 프로젝트 한 장이 따로 있습니다 — 업무 산출물이 아닌, 개인적으로 만들어 본 것들을 구분해 두었습니다.

한눈에 보는 요약

도메인은 자동차 → 공공·교육 → 산업 AI로 바뀌었지만, 일하는 방식은 늘 같았습니다. 체계를 세우고, 반복을 자동화하고, 필요한 도구를 직접 만들었습니다.

0 → 1

QA 프로세스 부재 환경에서 워크플로우 · 배포 게이트 · RCA 체계를 직접 설계

1일 → 한 줄

MLOps 릴리스 회귀 검증을 무인 실행으로 — E2E 시스템 1인 설계·구축·운영

전수 분석

누적 버그를 표본이 아닌 전수로 리뷰 → 10개 근본원인 분류 체계 수립 및 자동 축적화

80 %+

Playwright 도입으로 수동 회귀 테스트 시간 단축

ASPICE

Level 1 SWE.6 인증 획득 기여 (자동차 도메인)

경험한 도메인

자동차 임베디드 / HILs

커넥티드카 텔레매틱스

자율주행

공공 · 교육 웹

산업 AI 비전 · MLOps

LLM 기반 시스템

핵심 역량

프로세스 설계 — 정착과 구축

ASPICE Level 1 SWE.6 인증 환경에서의
구조적 QA 수행과, 프로세스 부재 환경에서
이슈 트래커 워크플로우 · Git 브랜치 전략 ·
자동화 규칙을 **0에서 설계**한 경험을 모두 보유.

테스트 자동화 · 회귀 검증

Playwright · Node.js 기반 MLOps E2E
회귀 시스템(실서버 3-worker 병렬) 1인 설계·
구축·운영, **팀 셀프서비스 실행 러너 구축**,
Selenium UI 자동화, Postman API 검증,
Python·Shell 기반 KPI·로그 수집 자동화,
Jenkins · GitHub Actions 연계.

자동차 임베디드 검증

HILs 기반 IVI 시스템 검증, CAN 통신
·CANoe 기반 네트워크 검증, 자율주행 제어
애플리케이션 검증, **KOLAS 공인 신뢰성 검사
통과**.

웹 품질 · 보안 점검

KRDS 웹 접근성 검증, REST API 검증
과정에서 **서버 측 검증 로직 부재를 지적하고
개선 주도**, Burp Suite · Lighthouse 기반 웹
보안·품질 진단.

AI 활용 · 도구 직접 제작

Claude Agent 기반 Regression TC
Creator(ClickUp · Google Sheets MCP
연동), Slack Slash Command + AWS
Lambda 기반 QA Request 봇, **Google
Apps Script 기반 QA Test Run 관리 도구**,
OCR 매크로 · KPI Calculator 등 실무
유틸리티 직접 개발.

협업 · 문서화

Jira · Jira Xray · Jira Automation ·
ClickUp · Confluence 기반 테스트 관리,
애자일 스크럼 운영, **월간/분기 품질현황보고
경영진 보고 경험**.

01

CHAPTER

QA 프로세스 & 워크플로우 설계

“표준이 있는 곳에선 정착시키고, 없는 곳에선 만든다.”

자동차 도메인에서는 ASPICE Level 1의 정형화된 산출물 체계를 따라 검증 절차를 운영했고, 현재는 **QA 프로세스가 부재한 환경에 워크플로우 · Priority 기준 · Root Cause 의무화 · 상태 머신을 직접 설계**해 다수 고객사 배포 QA를 동시 운영하고 있습니다.

개발·QA 워크플로우 설계

배포 게이트 · NO-GO 차단권

RC 버전 검증 절차

근본원인 분류 체계 · RCA

한눈에 보는 개선 결과

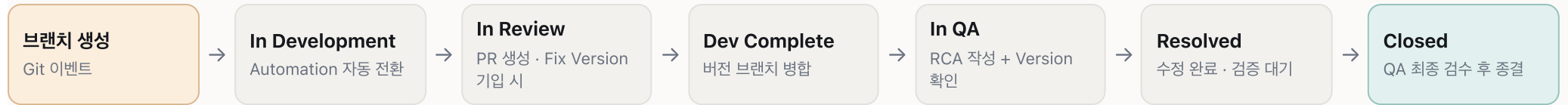
Git 브랜치 전략, 이슈 트래커 워크플로우, Automation 규칙을 하나의 시스템으로 직접 설계했습니다. 코드의 흐름(브랜치)과 일의 흐름(티켓 상태)을 연결해 수동 상태 전환과 누락된 맥락을 제거했습니다.

영역	기존 운영 방식	내가 도입한 변화
브랜치 운영	명명 규칙 부재, 생성/삭제 시점 모호	4종 브랜치 정의 + 네이밍·생애주기 규칙 수립
Priority 판단	작성자 주관에 의존	4단계 판단 Flow + 핵심 기능 목록 설계
티켓 유형	Bug/Task 혼재, 통계 불가	Task Type 분리 + Bug 전용 자동화 연결
결함 기록	“수정 완료”만 남고 원인 유실	Root Cause / Corrective Action 의무 작성
상태 전환	수동 전환, 누락·역행 빈번	Git 이벤트 → 트래커 상태 자동 전환
QA 진입 조건	구두 확인 필요	Fix Version·RC Version 기입 조건으로 자동화
알림	개별 확인 필요	상태 변경 시 Slack·이메일 자동 발송

본 문서는 실제 업무 경험을 바탕으로 재구성·가공되었으며, 회사 기밀 보호를 위해 모든 민감 정보는 익명화 예시 패턴으로 대체되었습니다.

설계한 전체 흐름

브랜치 동작이 어떻게 티켓 상태를 끌어오는지, 사람이 손대지 않아도 흐름이 유지되도록 연결했습니다.



마지막 문을 QA가 닫는다

Resolved 는 개발이 “고쳤다”고 선언한 상태일 뿐입니다. QA가 최종 검수한 뒤에만 Closed 로 넘어갑니다 — 티켓을 여는 사람과 닫는 사람을 분리해, 고쳤다는 말과 고쳐졌다는 확인이 섞이지 않게 했습니다.

되돌아가는 길도 설계

검증 반려 시 In QA → In Development 로 복귀.
RCA 미작성 상태로 Dev Complete 를 시도하면 Automation이 자동으로 In Development로 복구합니다.

운영 정착을 위해 함께 정한 규칙

- 모든 작업은 별도 브랜치에서 진행 (메인/버전 브랜치 직접 커밋 금지)
- PR 전 Fix Version 필드 반드시 기입 — 누락 시 Automation 미동작
- 통합 브랜치명에 Fix Version 문자열 포함 필수
- Task Type 누락·오분류 티켓은 QA 리뷰 단계에서 교정

QA 프로세스 설계

이슈 트래커 워크플로우

Git 브랜치 전략

Automation

Git 브랜치 전략

코드가 흐르는 길을 표준화

브랜치 명명 규칙이 없어 같은 종류의 작업이 서로 다른 패턴으로 만들어졌고, 삭제 시점도 모호해 사라지지 않는 브랜치가 누적되고 있었습니다.

정의한 브랜치 4종

유형	용도	유지
메인	전체 개발 기준, 개발 서버 배포용	영구
버전	기능 통합 · 특정 버전 배포 준비	배포 후 삭제
기능	티켓 단위 작업 (기능·버그·작업)	병합 후 삭제
핫픽스	배포 후 운영 이슈 긴급 수정	수정 후 삭제

네이밍 규칙

- 기능 — [티켓번호]_[간단한 기능 설명]
- 버전 — [버전번호]_[생성일자(YYMMDD)]
- 핫픽스 — [버전번호]_hotfix_[생성일자(YYMMDD)]

생애주기

브랜치	어디서 생성	언제 삭제
기능	버전 브랜치에서	버전 브랜치 병합 완료 시
버전	메인 브랜치에서	메인 병합 + 태그 생성 후
핫픽스	메인 브랜치의 태그에서	메인 병합 + 태그 생성 후

설계한 Workflow Step

- 기능 작업 — 티켓에서 기능 브랜치 생성 → 버전 브랜치로 PR (Fix Version 기입)
- 버전 준비 — 여러 기능 브랜치가 버전 브랜치로 병합
- 배포 준비 — 버전 → 메인 PR → 배포 직후 메인에 버전 태그 생성
- 핫픽스 — 태그된 메인에서 분기 → 수정 후 메인 병합 → 새 태그 생성

Priority 기준을 만들었다

문제 인식 — 작성자마다 Urgent/High 기준이 달라 우선순위 자체가 신뢰받지 못했습니다. “내 티켓은 다 Urgent”가 반복되는 상황이었습니다.

4단계 판단 Flow — 누가 작성해도 같은 결과가 나오도록



핵심 기능 목록(제품 범위 변경 시 갱신) — 검사 실행 및 판정 · 판정 결과 조회 · 모델 학습/배포 · 사용자 인증 · 데이터 저장 및 조회

등급별 처리 SLA — Priority를 단순 라벨이 아니라 실제 처리 흐름으로

등급	약속한 처리 방식
Urgent	발생 즉시 Slack 전체 공유 → 즉시 착수 → 4h 미해결 시 PM/TL 에스컬레이션
High	데일리 스크럼에서 리뷰 → 즉시 배정 → 현 스프린트 내 해결
Medium	백로그 등록 → 차기 스프린트 플래닝 시 반드시 포함
Low	백로그 등록 → 스프린트 여유 시 처리 (포함 보장 없음)

Bug/Task 분리 & 근본 원인 기록 의무화

2-2. Bug와 Task를 분리했다

문제 인식 — 버그와 신규 개발이 한 폴에 섞여 있어 “현재 미해결 버그 몇 건?”이라는 기본 질문에 즉답할 수 없었습니다. 자동화 규칙을 만들어도 일괄 적용밖에 안 돼 정교한 정책이 불가능했습니다.

내가 한 일 — Task Type 필드를 필수화하고 발생 경로별 분류 기준을 명문화. 이 분리 덕분에 **Bug 타입에만 RCA 자동화를 연결**할 토대가 마련됐습니다.

발생 경로	Task Type
QA 테스트 중 발견된 결함	Bug
운영 환경에서 리포트된 이슈	Bug
기획서 기반의 신규 개발·개선	Task

분류 책임은 티켓 생성자에게 두되, **QA 리뷰 단계에서 오분류를 확인·교정**하는 흐름까지 함께 설계했습니다.

2-3. 근본 원인 기록을 의무화했다

문제 인식 — 코드만 고치고 끝나면 왜 발생했는지와 어떻게 해결했는지의 맥락이 유실되어, 같은 버그가 다른 모듈에서 재발하는 패턴이 반복되고 있었습니다.

내가 한 일 — Root Cause / Corrective Action 필드를 신설하고, **Dev Complete 전환 시 작성을 강제하는 Automation**을 구성했습니다.

미작성으로 상태를 넘기려 하면 **자동으로 In Development로 복구**됩니다.

작성 원칙 — “수정 완료” 식 기록 차단

- Root Cause** — 현상이 아닌 원인을 기술. 코드 위치(파일/함수) 명시. “확인 누락” 같은 모호한 표현 금지.
- Corrective Action** — 변경 내용, 영향 범위(사이드 이펙트 가능 영역), 관련 PR/커밋 링크.

기대 효과 — QA는 RCA를 기반으로 **타겟 검증 시나리오**를 설계할 수 있어 전체 회귀 의존도가 줄어듭니다. 축적된 RCA 패턴은 스프린트 회고에서 프로세스 개선 액션의 근거가 됩니다.

상태를 간소화하고 Fix Version을 도입했다

문제 인식 — Dev Complete → Ready for QA → In QA 3단계는 형식적으로만 운영되고 있었고, 배포 후 Done 단계는 사실상 사문화 상태였습니다. 어떤 버전에 어떤 이슈가 들어갔는지 확인하려면 매번 개발자에게 구두로 물어야 했습니다.

▼ 중복 단계 통합

Dev Complete + Ready for QA + In QA

→ **Dev Complete** → **In QA** 2단계로 축소

Ready to Release + Released + Verified

→ **Resolved** 하나로

NEW 필드 · 상태 신설

Fix Version · **RC Version** 추가 → 버전별 이슈 이력 추적 가능

No Issue · **On Hold** · **Won't Do** — 이슈가 어떤 이유로 닫혔는지 구분 가능

Closed — QA 최종 검수를 통과한 티켓만 도달하는 종착 상태. **Resolved** (수정 완료)와 분리해 **종결 권한을 QA에 둔 것**이 핵심입니다.

연결한 정책 — Dev Complete 티켓 중 **Fix Version·RC Version**이 모두 기입된 것만 In QA 전환 대상으로 인정하도록 자동화에 연결했습니다. 구두 확인이 필요했던 “이거 QA 봐도 되나요?”가 필드 조건으로 치환됐습니다.

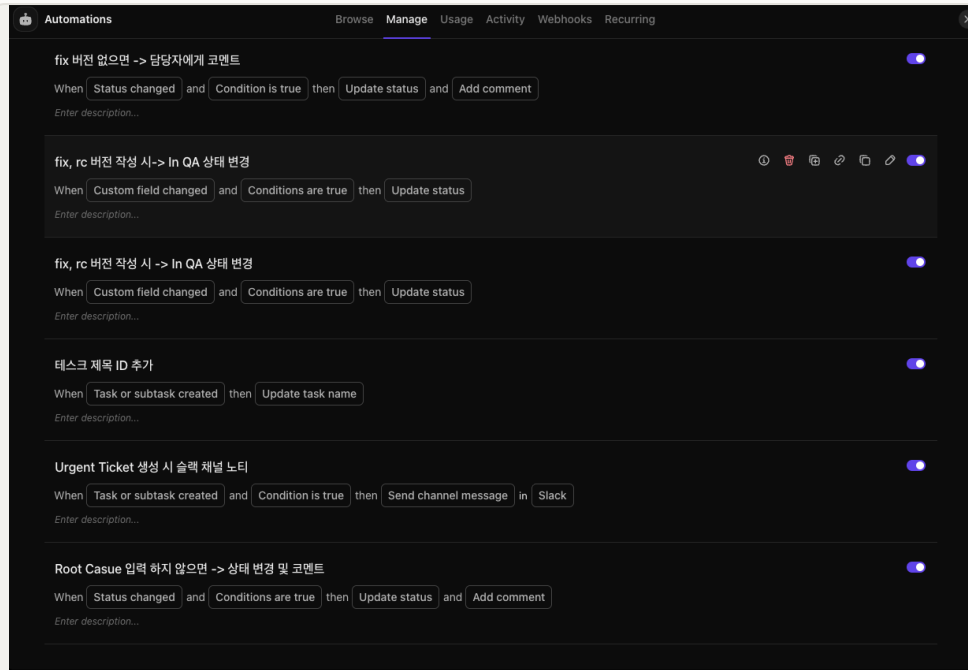
Automation

Git과 이슈 트래커를 하나로 묶었다

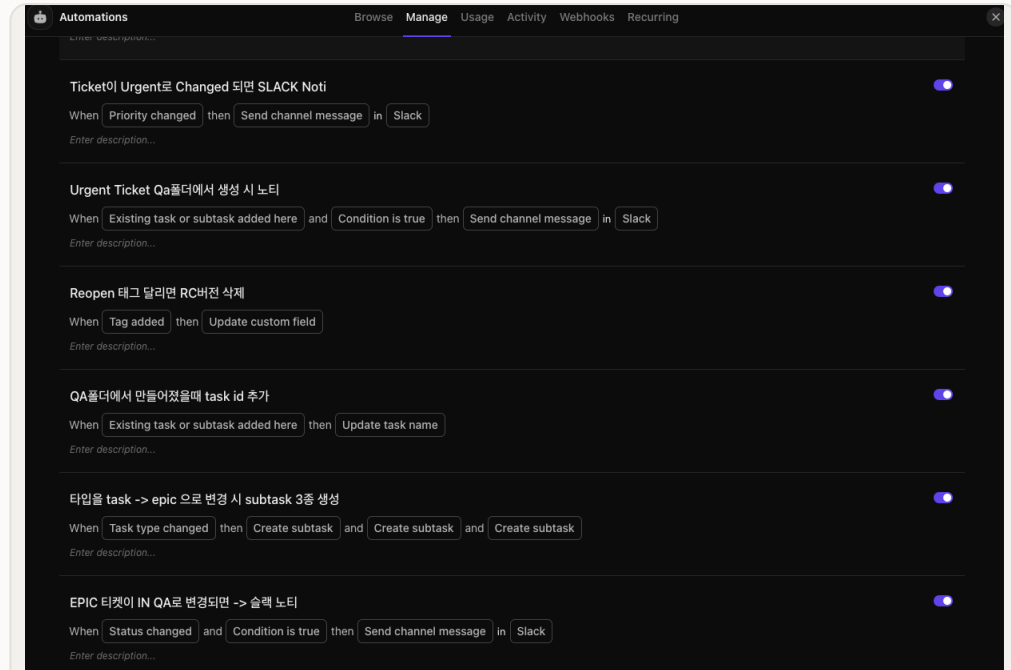
두 영역을 따로 설계해놓는 것만으로는 의미가 적습니다. 사람이 매번 상태를 옮겨야 하기 때문입니다. **Git 이벤트가 발생하면 트래커 상태가 따라오도록, 그리고 정책을 우회한 티켓은 자동 차단되도록** 구성했습니다.

트리거	조건	자동 동작
브랜치 생성	담당자가 할당된 티켓의 기능 브랜치	상태 → In Development
PR 생성	Fix Version 필드 기입됨	상태 → In Review
PR 병합	버전 브랜치명에 Fix Version 포함됨	상태 → Dev Complete
상태 → Dev Complete	Task Type=Bug AND RCA 미작성	상태를 In Development 로 자동 복구
상태 → In QA	Fix Version OR RC Version 미기입	전환 차단
상태 → In Progress / Reopen	—	시작일 · 종료일 자동 세팅
티켓 생성	요약 키워드 매칭	연관 이슈 자동 Relates to 연결
티켓 생성	관련자 자동 식별	Watcher 자동 추가

실제 구성 화면 — 트래커에 등록된 규칙들



상태 · 커스텀 필드 변경을 트리거로 상태 전환과 코멘트를 자동 실행한다. 맨 아래가 Root Cause 미입력 시 상태를 되돌리는 규칙.



우선순위 · 태그 · 타입 변경까지 트리거로 활용한다. Urgent 승격 즉시 Slack 공지, Epic 전환 시 하위 작업 자동 생성.

앞의 표와 대조해 보면

- Root Cause 입력 하지 않으면 → 상태 변경 및 코멘트 — **RCA 미작성 시 되돌리는 규칙**이 실제로 걸려 있습니다
- fix, rc 버전 작성 시 → In QA 상태 변경 — **버전 필드를 QA 진입 조건으로** 삼은 정책이 자동화로 구현돼 있습니다
- Ticket이 Urgent로 변경되면 SLACK Noti — Priority 등급별 SLA에서 약속한 **“Urgent 발생 즉시 전체 공유”**가 사람이 아니라 규칙으로 보장됩니다

화면은 운영 중인 규칙 목록의 일부입니다. Epic 전환 시 하위 작업 3종 자동 생성, Reopen 태그 시 RC 버전 초기화, 폴더 기준 티켓 ID 자동 부여 등 **티켓 생애주기 전반에 걸쳐** 손이 가던 지점들을 규칙으로 옮겼습니다.

상태 변경(To Do → In Progress , Dev Complete , Done 등) 시 관련자에게 **Slack·이메일 알림이 자동 전송**되도록 연결했습니다. 팀원 간 상태 확인 비용을 제거하고 의사결정 속도를 높이는 것이 목적이었습니다.

이 작업이 보여주는 것 QA 한 단계 자동화 구현이 아니라 **상황별 조건 분기를 설계**할 수 있다는 점 (Task Type, RCA 작성 여부, Version 필드 등 다중 조건의 조합).

트래커 밖의 요청까지 시스템 안으로

Part 3까지가 트래커 내부 자동화였다면, 여기서는 한 단계 더 나아갔습니다. **슬랙-서버리스-트래커를 직접 연결하는 봇을 구현해 배포**하고, 사용자 가이드까지 작성해 사내에 확산시켰습니다.

문제 인식

여러 프로젝트의 QA 요청이 각자 다른 채널·메신저·구두 전달로 들어와 **요청 자체가 일원화되지 않은 상태**였습니다. 요청 시점·범위·일정이 누락된 채 들어오기 일쑤였고, QA 입장에서 “무엇을 언제까지 어떤 버전으로 검증해야 하는지”를 매번 되묻는 비용이 발생했습니다.

구조

/qa-request

Slack Slash Command — 아무 채널·DM에서



AWS Lambda

요청 핸들러 → Block Kit 모달 표시



AWS Lambda

제출 핸들러 → 검증 → ClickUp API



QA 티켓 생성



티켓 제목

ex) Daisy Pro 1.9.0 QA 요청

프로덕트 (프로젝트)

프로덕트 선택

테스트 목적

ex) 1.9.0 배포를 위한 QA 요청

테스트 버전 (옵션)

ex) 1.9.0

💡 버전은 숫자.숫자.숫자 형식으로 입력해주세요. ex) 1.9.0

변경 범위

ex)
데이터셋 비디오 기능 추가
모델 개발 파이프라인 기능 추가

취소

생성

슬랙에서 /qa-request 입력 시 뜨는 요청 양식. 제목 · 프로덕트 · 목적 · 버전 · 변경 범위 · 일정 · 배포 예정일 · 고객사 배포일을 받는다.

Slack API

Slash Commands

Block Kit Modal

AWS Lambda x2

ClickUp API

Python



티켓 생성 + 채널 공지

이후 모든 알림이 이 메시지 스레드에 축적

스레드를 앵커로 삼았다 — 생성 알림 메시지 하나를 기준으로 두고, 그 티켓과 관련된 **일정 변경·배포 리마인드가 전부 같은 스레드에** 달리게 했습니다. 채널이 알림으로 도배되지 않으면서도 하나의 요청에 대한 맥락이 흩어지지 않습니다.

입력 설계 — 잘못 낸 요청이 통과하지 못하게

요청 누락을 규칙으로 부탁하는 대신 **양식이 강제하도록** 만들었습니다.

제출 전에 막는다

버전은 X.Y.Z 형식만 허용하고, 어긋나면 **실시간으로 경고**가 뜨며 제출 자체가 막힙니다. v1.9.0 의 v 는 자동 제거하고, 1.9 · 1.9.0-RC1 은 거부합니다.

테스트 버전 (옵션)

● **버전 형식이 올바르지 않습니다.** 숫자.숫자.숫자 형식으로 입력해주세요. ex)

1.9.0

형식이 맞지 않으면 실시간 경고가 뜨고 제출이 막힙니다.

왜 이렇게

버전은 뒤에 이어지는 **배포 리마인더의 판정** 키입니다. 여기서 형식이 틀리면 자동화 전체가 조용히 동작하지 않으므로, **입구에서 막는 것이 가장 싸다고** 봤습니다.

필요할 때만 나타난다

프로덕트는 드롭다운에서 고르되, 목록에 없으면 **“직접 입력”을 선택했을 때만** 텍스트 칸이 추가됩니다. 자주 쓰는 제품은 관리자가 ClickUp 옵션으로 등록해 다음부터 드롭다운에 나타나게 했습니다.

테스트 버전 배포 예정일

시간대: 서울

🔔 이 시각까지 QA 버전(RC1)이 배포되지 않으면 리마인더 알림이 발송됩니다.

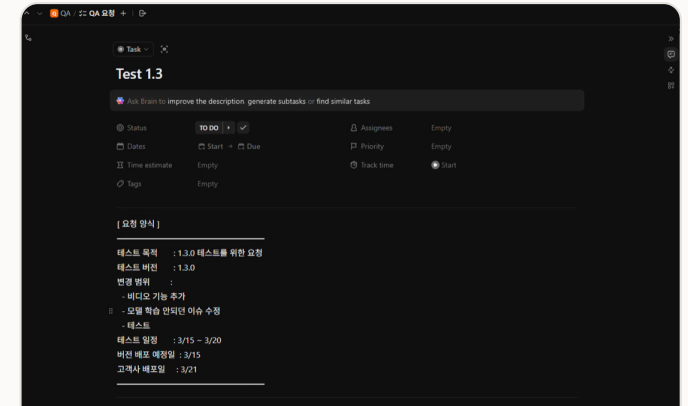
배포 예정 시각. 이 값이 리마인더의 기준점이 된다는 설명을 입력 칸 바로 아래에 붙였다.

왜 이렇게

배포 예정 시각을 굳이 받는 이유를 **입력 칸 아래에 그 자리에서 설명**했습니다. 무엇에 쓰이는지 모르는 입력은 아무렇게나 채워지고, 그러면 리마인더가 신뢰를 잃습니다.

제출하면 이렇게 남는다

ClickUp에 티켓이 자동 생성되고, 본문은 **[요청 양식] 블록으로 구조화**됩니다. 시작일/마감일과 커스텀 필드(프로덕트 · 테스트 버전 배포일 · 고객사 배포 예정일)까지 함께 채워집니다.



제출 즉시 생성된 ClickUp 티켓. 본문이 [요청 양식] 블록으로 정리되고 날짜·커스텀 필드가 자동 입력된다.

티켓을 만드는 데서 끝내지 않았다

요청서를 받는 봇은 혼합니다. 여기서 한 걸음 더 나가, 요청에 적힌 약속이 실제로 지켜지는지를 봇이 계속 확인하도록 만들었습니다.

17 일정이 바뀌면 자동 공지

누군가 ClickUp에서 Start / Due Date를 바꾸면 **스레드에 변경 전·후가 자동으로** 달리고 요청자와 담당자가 멘션됩니다.

QA Ticket Bot 오픈 오늘 오후 4:47
17 Start Date / Due Date 변경 @김상욱(마케팅/기획 - QA) @
- 티켓: test 9.9.9
- 변경자: 김상욱
- 2026-07-09 (목) ~ 2026-07-11 (토) → 2026-07-14 (화) ~ 2026-07-17 (금)

일정 변경 시 관련자 멘션과 함께 변경 전 → 후가 스레드에 자동 공지된다.

운영 규칙으로 “변경하면 흔적을 남겨 달라”고 부탁하던 것을 봇이 대신하게 바꿨습니다.

✅ 배포되면 조용히 닫는다

봇이 `release` 채널의 **Jenkins 빌드 메시지를 감시**하면서, 티켓에 적힌 버전의 RC1 빌드가 올라왔는지 스스로 확인합니다. **예정 시각보다 일찍 배포돼도** 확인되는 즉시 알립니다.

QA Ticket Bot 오픈 오늘 오후 4:31
✅ QA 티켓이 생성됐습니다!
요청자:
제목: 검증 버전 배포 성공
프로덕트: Daisy Pro
테스트 일정: 7/9 (목) ~ 7/10 (금)
ClickUp 링크: <https://app.clickup.com/>

1개의 댓글

QA Ticket Bot 오픈 오늘 오후 4:31
✅ QA 버전 배포 확인됨
- 티켓: 검증 버전 배포 성공
- Daisy Pro 1.7.0 → 1.7.0-RC1 빌드 성공 확인 (예정 시각 전 배포)

예정 시각 전에 빌드가 확인되면 “배포 확인됨”으로 스스로 닫는다.

⚠️ 안 되면 계속 두드린다

예정 시각이 지나도 빌드를 찾지 못하면 **채널에 경고**가 뜨고 관련자가 멘션됩니다. 이후 **2시간 간격으로 재확인**하며, 배포가 확인되면 ✅ 알림 후 종료합니다.

QA Ticket Bot 오픈 오늘 오후 4:23
✅ QA 티켓이 생성됐습니다!
요청자:
제목: QA 버전 배포 성공 케이스
프로덕트: Daisy Pro
테스트 일정: 7/9 (목) ~ 7/10 (금)
ClickUp 링크: <https://app.clickup.com/>

2개의 댓글

채널에도 전송됨
QA Ticket Bot 오픈 오늘 오후 4:25
⚠️ QA 버전이 배포되지 않았습니다! @김상욱(마케팅/기획 - QA)
- 티켓: QA 버전 배포 성공 케이스
- 프로덕트: Daisy Pro / 버전: 1.7.0
- 배포 예정: 7/8 (수) 16:25
- release 채널에서 DAISY-PRO-RC 1.7.0-RC1 빌드 성공을 찾지 못했습니다. (배포 확인까지 2시간 간격으로 다시 알립니다)

예정 시각까지 빌드를 찾지 못하면 경고 + 관련자 멘션. 배포될 때까지 2시간 간격으로 재알림.

왜 이렇게

“QA 버전 언제 나와요?”를 사람이 묻는 일 자체를 없애려고 했습니다. **지켜지지 않은 약속을 사람이 발견하는 구조에서, 시스템이 먼저 알리는 구조로 옮긴 것**입니다.

알림 시스템의 진짜 난이도는 “언제 안 보낼지”에 있다

자동 알림은 만들기는 쉽고 **미움받기도 쉽습니다**. 팀이 봇을 끄지 않게 만들기 위해 정의한 경계 조건들입니다.

상황	순진하게 만들면	실제 설계
날짜 최초 설정	변경으로 보고 알림 발송	알림 없음 — 변경·삭제일 때만
시작일·마감일 동시 변경	알림 2건	1건으로 병합
예정 시각 전 배포	예정 시각까지 대기	확인 즉시  처리
미배포 지속	1회 경고 후 방치	2시간 간격 재알림
계속 안 나옴	무한 반복 → 알람 피로	24시간 후 자동 종료
배포일이 변경됨	옛 시각 기준으로 오탐	1분 내 감지 → 새 기준 재동작
티켓이 삭제됨	고아 리마인더가 계속 울림	1분 내 자동 정리
판정 규칙 없는 제품	오탐 알림	티켓만 생성 · 리마인더 미동작

도구와 함께 사용자 가이드를 만들었다 — 명령어 입력부터 자동 알림 해석, 자주 묻는 질문 (버전 형식 오류 · 직접 입력 제품 · 알람이 계속 올 때)까지 **문의가 들어올 지점을 먼저 답해** 문의 자체를 줄였습니다.

이 봇이 만든 변화

항목	도입 전	도입 후
요청 진입점	DM · 구두 · 여러 채널 산재	슬래시 명령 하나로 일원화
필수 정보 누락	빈번 (버전·일정·범위)	양식이 강제 + 형식 검증
티켓 생성	수동 (QA가 받아 적어 입력)	제출 즉시 자동 생성 · 필드까지 채움
일정 변경 추적	사람이 흔적을 남겨야 함	봇이 자동 공지 · 멘션
QA 버전 배포 확인	“언제 나와요?”를 사람이 물어봄	봇이 빌드를 감시해 먼저 알림
팀 가시성	요청 사실 자체가 묻힘	채널 공지 + 스레드에 이력 축적

이 작업이 보여주는 것

- 규칙을 만드는 사람에서 나아가 **규칙이 동작하도록 시스템을 직접 빌드**
- Slack · 서버리스 · 트래커 API · CI 알림이라는 **이질적 시스템을 하나의 경험으로 연결**
- **실패 케이스와 노이즈까지 설계 범위에 포함** — 알림은 “보내는 것”이 아니라 “언제 안 보낼지”를 정하는 일

QA의 NO-GO 차단권

미검증 빌드가 고객사 현장에 배포된 인시던트를 계기로, “무엇이 배포되는가”를 사람의 기억이 아닌 프로세스로 보증하는 배포 게이트를 설계·수립했습니다.

배경 — 인시던트에서 출발, 원인은 개인의 실수가 아니라 구조의 공백

- 배포 대상 확인이 **버전 라벨 기준**으로만 이루어져, 같은 버전 라벨 아래 서로 다른 빌드가 존재할 수 있는 상황을 걸러내지 못함
- **QA 검증 완료 여부**와 **배포 결정** 사이에 명시적인 **연결 고리가 없음** — 검증이 끝나지 않아도 배포를 막는 장치가 부재

설계 — 배포 전 4단계 게이트



본 문서는 회사 보안 정책에 따라 고객사 버전 정보를 익명화한 요약본입니다.

권한 분리와 예외 레인

🚫 QA의 NO-GO 차단권

미검증 빌드에 대해 QA가 **NO-GO**를 선언해 배포를 차단할 수 있는 권한을 프로세스에 명문화했습니다.

동시에 역할을 분리했습니다 — QA는 품질 상태를 보고하고 차단권을 행사하며, **go/no-go의 최종 결정 권한은 PM에게** 둡니다.

품질 보고 책임과 배포 결정 권한이 섞이지 않도록 한 설계입니다.

🚨 비상 배포 예외 레인

긴급 배포가 필요한 현실을 부정하는 프로세스는 **우회됩니다**. 예외 상황을 프로세스 밖이 아니라 **안으로** 끌어들이었습니다.

- **위험 감수 배포에 대한 명시적 승인** — 누가 어떤 위험을 감수하기로 했는지 기록
- **미검증 범위 문서화** — 무엇이 검증되지 않은 채 나가는지 명시
- **사후 QA 의무화** — 배포 후 검증으로 공백을 닫음

의미

- 인시던트를 **책임 추궁이 아닌 프로세스 개선**으로 전환 — 품질현황보고에서도 책임 소재 없이 사실과 개선 방향 중심으로 공유
- “검증됐는가”를 개인의 기억·선의에 의존하지 않고 **체크 가능한 게이트**로 치환
- QA가 릴리즈 파이프라인의 형식적 참여자가 아니라 **명시적 권한을 가진 게이트키퍼**로 자리매김

배포 게이트

Release Management

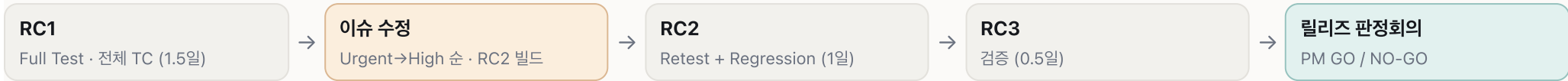
NO-GO 차단권

빌드 아티팩트 검증

인시던트 기반 개선

Exit Criteria와 반복 가능한 절차

RC 검증이 “상황마다 다르게” 진행되던 것을, **Exit Criteria**와 역할 분리가 명문화된 반복 가능한 절차로 정립했습니다.



도식은 QA 5일 가정의 대표(최대) 시나리오 예시이며, RC 개수·날짜는 기간에 따라 스케일합니다.

Exit Criteria — 최종 RC가 전부 충족하면 GO 후보

0건
잔여 Urgent (우선 게이트)

< 5건
잔여 High (건별 PM·PL 협의)

≥ 95 %
Pass Rate

Known
Normal/Low는 개별 게이트 없이 Pass Rate에 포함

개별 건을 놓고 매번 논쟁하는 대신, **숫자로 판정 가능한 기준**을 정의해 판정 비용을 줄였습니다. “언제 끝나는가”를 감이 아니라 **Exit Criteria 충족 여부**로 답할 수 있게 됐습니다.

배포 리듬 · 역할 분리 · Hotfix 트랙

수정 파이프라인 & 배포 리듬

- Priority 순 착수 (Urgent → High → 그 외), **검증 중에도 다음 RC 수정 병행**
- 배포 시점 **완료분만 탑재, 나머지 이월** — Regression은 직전 RC 대비 전체 머지분 대상
- 준비되는 즉시 배포(당일도 가능) — **1영업일은 상한**이지 고정 간격이 아님
- 5일 기준 실질 RC 최소 3회 (빠르면 그 이상), RC 개수는 **상한 없이 Exit Criteria 충족까지** 반복

판정 — 역할 분리

역할	책임
QA	Exit Criteria 충족 여부 보고만 — 품질 상태를 판정 가능한 형태로 전달
PM	최종 릴리즈 GO / NO-GO 결정 — 미충족 시 기간 연장이나 Known Issue GO도 PM 판단

품질 보고 책임(QA)과 배포 결정 권한(PM)을 분리해, QA가 비즈니스 판단을 떠안지 않고 **품질 사실에 집중**할 수 있게 했습니다.

별도 트랙 — Hotfix (프로덕션 긴급 수정 · 1.X.Y)

프로덕션 배포 후 발견된 Urgent는 Full Test 없이 **자동화 기반 빠른 검증**만 거칩니다.



정규 레인과 긴급 레인을 분리해, **긴급 수정이 정규 절차를 무너뜨리지 않으면서도** 자동화 안전망 안에서 빠르게 나갈 수 있게 했습니다.

의미 — 배포 게이트(무엇이 나가는가)와 RC 절차(어떻게 수렴하는가)가 짝을 이뤄 릴리즈 파이프라인 전체를 커버합니다.

검증 결과를 흩어진 시트가 아니라 하나의 이력으로

Google Sheets + Apps Script로 **Test Run 생성 · 실행 · 완료 기록**을 자동화하고, 모든 결과를 단일 History에 축적하는 도구를 직접 만들었습니다.

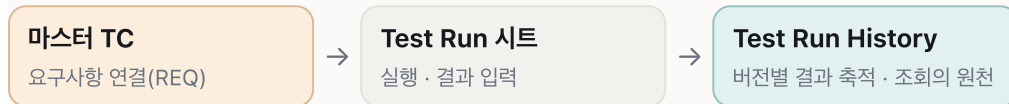
문제 — 한 방향으로만 보이는 데이터

기존에는 버전(RC)마다 마스터 시트에서 TC를 통째로 복사한 뒤 필요 없는 항목을 수동으로 지우고, 실행 결과를 각 버전 시트에 흩어서 기록했습니다.

보고 싶은 것	기존
버전 → TC “이번 버전에 뭐가 있나”	보였음
TC → 버전 “이 TC가 버전을 거치며 어떻게 됐나”	여러 시트를 눈으로 따라가야 함

회귀 대상 선별 · 버전별 요약 · 추적성 근거도 **매번 수동**이었습니다.

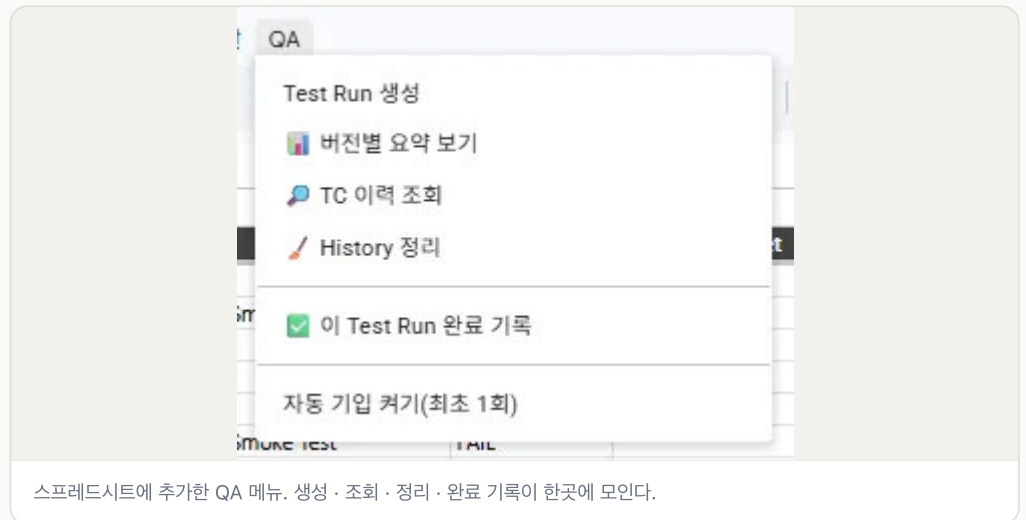
설계 원칙 — 쌓는 곳과 보는 곳의 분리



History는 결과를 **세로로 계속 쌓는 원천**이고, 사람은 그 위에 얹은 **조회 뷰**(버전별 요약 · TC 이력)로 봅니다. 데이터베이스 원본 테이블을 직접 뒤지지 않고 **쿼리로 조회하는 것과 같은 구조**입니다.

QA 메뉴 구성

메뉴	기능
Test Run 생성	마스터/기존 시트에서 조건 필터로 새 Test Run 시트 생성
버전별 요약	버전별 총 TC / PASS / FAIL / NA / Pass Rate 집계
TC 이력 조회	특정 TC의 버전별 결과 변천사
History 정리	삭제되어 빈 줄로 남은 이력 행 정리
완료 기록	현재 시트 결과를 History에 확정 기록
자동 기입 켜기	실행일·실행자 자동 기입 트리거 설치 (최초 1회)



확산까지 설계했다 — 자동 가입 트리거는 **스프레드시트 자체에 설치**되어 누가 편집하는 동작합니다. **팀원이 각자 켤 필요가 없습니다.** 도구를 나눠주는 것과 팀이 실제로 쓰게 만드는 것은 다른 문제라고 봤습니다.

회귀 대상 선별과 조회를 손에서 떼어냈다

필터로 Run을 만든다

컬럼(타입 · 기능영역 · 세부영역 · 모델타입 · 우선순위 · 레이블 · P/F)을 골라 값을 선택합니다. 한 필터 안 여러 값은 **OR**, 두 번째 필터부터는 앞과 **AND/OR**로 결합합니다. 같은 컬럼을 AND로 두 줄 쓰면 둘 다 가진 TC(예: Smoke + Regression)만 뽑힙니다.

회귀 승계 — RC2를 만드는 법

소스를 기존 **Test Run 시트**에서 RC1로 두고 필터를

P/F = FAIL, NA 로 걸면, **RC1에서 깨졌거나 미검증인 항목만** P/F 빈 채로 승계됩니다.

RC 검증 절차의 “Retest + Regression”을 손이 아니라 도구가 계산합니다.

생성 시 P/F·Defect·비고는 빈 상태로, 실행일·실행자 컬럼이 추가되고, 통계 블록과 요구사항 링크(REQ)는 보존됩니다.

실행 — 기록을 사람이 하지 않는다

- P/F 드롭다운 입력 시 같은 행의 **실행일(KST)·실행자가 자동 기입**
- P/F를 지우면 실행일·실행자도 **함께 비워짐**
- 실행자는 **이름매핑 시트**(이메일 → 이름)를 참조해 이름으로 기록

완료 기록 — 확정과 갱신

- 결과가 History에 기록 (**미실행 행 제외**)
- 완료 시트는 이름에  ... - 완료 , 탭 색 초록으로 표시
- 같은 Run을 재기록하면 기존 이력을 최신값으로 갱신 — 재검증 FAIL→PASS가 반영됨

왜 이렇게

누가 언제 검증했는지를 **사람의 성실함에 맡기면 반드시 비어 있습니다**. 입력의 부산물로 자동으로 남게 만들어야 추적성이 실제 데이터가 됩니다.

조회 — 이 도구의 목적지

축적된 History 위에서 두 가지 조회를 제공하고, 결과는 **표 복사 버튼으로 보고서에 색상까지 그대로 붙여넣을 수** 있습니다. (보안 설정으로 막히면 탭 구분 텍스트로 풀백)

이 도구로 답할 수 있게 된 질문	답하는 방법
커버리지 — 이번 버전에 뭘 검증했나?	Test Run 시트
회귀 대상 — 과거 FAIL 난 TC는?	History 필터 / 시트
배포 판단 — 이 버전 Pass율·미해결 결함은?	 버전별 요약
추적성 — 이 TC가 버전마다 어땠나?	 TC 이력 조회

 버전별 요약

버전별 요약

표 복사닫기

버전	총 TC	PASS	FAIL	NA	Pass Rate(%)
TestRun_20260721 (2)	4	2	2	0	50
TestRun_20260721	4	1	2	1	33.3

 버전별 요약 — 총 TC · PASS · FAIL · NA · Pass Rate가 버전별로 자동 집계된다. 표 복사로 보고서에 그대로 붙인다.

TC 이력 조회

TC 이력 · TC_01

표 복사

닫기

버전	TC_ID	기능영역	우선순위
TestRun_20260721 (2)	TC_01	ADMIN	
TestRun_20260721	TC_01	ADMIN	

TC 이력 조회 — 하나의 TC가 버전을 거치며 어떻게 됐는지 세로로 쌓인 History에서 뽑아 보여준다.

다른 파트와 이어지는 지점

- **RC 검증 절차**의 Exit Criteria(Pass Rate 95%)를 이 도구가 자동 집계합니다
- **월간 품질현황보고**에 버전별 요약을 그대로 복붙합니다
- **Regression TC Creator**가 생성한 TC도 이 시트 체계 위에 있습니다

QA Test Run 관리 도구 사용 가이드

Google Sheets 기반 Test Run 생성 · 실행 · 이력 관리 자동화

Alpha Labs · QA · Daisy Platform | 작성: 김성욱

1. 개요

이 도구가 해결하는 문제

기존에는 버전(RC)마다 마스터 시트에서 TC를 통째로 복사한 뒤 필요 없는 항목을 수동으로 지우고, 실행 결과를 각 버전 시트에 붙여넣기 하곤 했습니다. 이 방식은 버전 → TC 방향만 보였고("이번 버전에 뭐가 있나"), 정작 TC → 버전 방향"이 TC가 버전을 거치며 어떻게 됐다"은 여러 시트를 눈으로 따라가야 했습니다. 회귀 대상 산별 버전별 요약·추적성 근거도 매번 수동이었습니다.

이 도구가 만드는 흐름

생성 → 실행 → 완료 기록의 3단계를 자동화하고, 모든 결과를 하나의 이력(History)에 축적합니다. 추적성의 핵심인 "요구사항 → TC → 실행 결과 → 결함"에서 결과 축을 실패이력으로 확보합니다.



핵심 원칙 — 열는 곳과 보는 곳의 분리

History는 결과를 세로로 계속 열는 원천(원본)이고, 사람은 그 위에 얹은 조회 뷰(버전별 요약 · TC 이력)로 봅니다. 데이터베이스 원본 테이블을 직접 뒤지지 않고 쿼리로 조회하는 것과 같은 구조입니다.

2. 본문

2-1. QA 메뉴 구성

메뉴	기능
Test Run 생성	마스터(기존 시트)에서 조건 필터로 새 Test Run 시트 생성
버전별 요약 보기	버전별 총 TC / PASS / FAIL / NA / Pass Rate 합업
TC 이력 조회	특정 TC의 버전별 결과 변천사 확인
History 정리	삭제되어 인 줄로 남은 이력 행 정리
이 Test Run 완료 기록	현재 시트 결과를 History에 기록(확장)

도구와 함께 작성한 사용자 가이드. 개요 · 메뉴 구성 · 최초 설정 · 생성/실행/완료/조회 · 유의사항으로 구성.

일회성 분석이 아니라, 원인이 쌓이는 구조를 만들었다

프로세스 제안을 “필요해 보여서”가 아니라 **데이터**로 하기 위해 누적된 버그를 전수 분석했고, 그 결과를 **한 번의 리포트로 끝내지 않고** 이후의 모든 버그가 같은 형식으로 자동 축적되는 체계로 바꿨습니다.

문제 — 원인이 데이터로 남지 않는다

버그 티켓은 쌓이는데, 근본 원인은 자유 서술로 흩어지거나 아예 기록되지 않았습니다. 그 결과:

- “어떤 유형의 원인이 반복되는가”를 아무도 **정량적으로** 답할 수 없음
- 프로세스 개선의 우선순위를 감(느낌)으로 정하게 됨
- 구조화된 Root Cause 필드를 도입하자고 제안해도, **설득 근거가 없음**

진행 — 표본이 아니라 전수를 봤다

- 이슈 트래커에 누적된 **버그 티켓 전수 리뷰** — 표본 추출이 아니라 전부
- 근본 원인 유형을 **귀납적으로 도출**해 10개 카테고리 분류 체계 수립
- 분류 체계와 실제 분포 데이터를 근거로 **구조화된 Root Cause 필드(선택형)** 도입 제안 → 채택

분류 체계를 먼저 정의하고 티켓을 끼워 맞추는 것이 아니라, **실제 티켓에서 반복 패턴을 추출**해 카테고리를 만들었기 때문에 현장의 결함 양상을 그대로 반영합니다.

전환 — 손으로 쓴 것을 자동으로 쌓이게

자유 서술 · 미기록

원인이 흩어져 집계 불가



전수 분석 (1회)

귀납적으로 10개 카테고리 도출



선택형 필드 + 작성 의무화

Dev Complete 전환 시 Automation이 강제



원인 분포가 자동 축적

이후 모든 버그가 스스로 데이터가 된다

그래서 “몇 건을 분석했는가”는 중요하지 않습니다. 분석 시점의 건수는 계속 늘어나고, 그것이 바로 이 체계가 의도한 결과입니다. 목적은 한 번의 정확한 집계가 아니라, **버그가 늘어나는 동안에도 원인 분포가 자동으로 최신 상태를 유지하는 것이었습니다.**

활용 — 이 데이터가 흘러가는 곳

- 품질현황보고의 **정량 근거** — 반복 원인 상위 카테고리 기준으로 개선 우선순위 도출
- 정기 트리아지 · Known Issue 백로그 관리의 기초 데이터
- 배포 기준 · Severity 정의 공식화 작업의 출발점
- **Regression TC Creator의 입력** — Agent가 이 필드를 읽어 회귀 TC를 설계 (Ch.03)

핵심 — “제안하기 전에 전수를 본다”는 접근. 필드 하나를 추가하는 작은 프로세스 변경도 데이터 근거 위에서 제안하면 조직의 수용 속도가 달라집니다.

Root Cause Analysis

결함 분류 체계

데이터 기반 프로세스 설계

품질 메트릭

02

CHAPTER

테스트 자동화

공간정보진흥원 프로젝트에서 **Playwright**를 처음 도입해 수동 회귀 검증을 자동화로 전환하며 테스트 시간 80% 이상을 단축했습니다. 이 경험은 이후 모든 QA 활동에서 **"자동화 가능 여부"**를 기본적으로 검토하는 사고방식으로 자리잡았고, 현재는 MLOps 파이프라인 전 구간을 무인 검증하는 E2E 회귀 시스템까지 확장되었습니다.

DAISY MLOps E2E 회귀 시스템

셀프서비스 테스트 러너

Playwright UI Automation

Selenium 반응형 검증 도구

사람이 하루 걸리던 릴리스 검증을 명령 한 줄로

비전 AI 학습 플랫폼(데이터셋 → 라벨링 → 학습 → 검증 → 배포)의 전체 파이프라인을 Playwright로 자동 검증하는 회귀 테스트 시스템을 1인으로 설계·구축·운영하고 있습니다. 아래 숫자는 커버리지가 아니라 이 시스템이 만들어낸 레버리지입니다.

1일 → 한 줄

릴리스 회귀 검증 · 무인 실행 + 자동 보고

3 worker

단일 공유 계정 실서버에서 병렬 실행 성립

타입당 1회

30분~3시간짜리 학습을 픽스처로 재사용

2 배 +

스위트가 늘어나는 동안 아키텍처는 그대로

항목	내용
역할	QA Engineer · 1인 설계 / 구현 / 운영
기간	2026.06 ~ (운영 중, 지속 확장)
스택	Playwright · Node.js · Slack API
대상	실서버 3개 환경 (cloud / ops / aws) — 목(mock) 없이 실제 파이프라인 검증
커버 범위	프로젝트 생명주기 · 데이터셋/라벨링 · 전처리 · 캔버스/ROI · 7종 AI 모델 학습~배포 · 모델 소비(내보내기·IoU) · 모델 성능 비교 · 회귀

TC 개수를 앞세우지 않는 이유 — 스위트는 현재 500여 개 TC 규모지만, 그 숫자만으로는 무엇이 얼마나 잘 검증되는지를 알 수 없습니다. 이 시스템에서 실제로 어려웠던 것은 개수를 늘리는 일이 아니라, 실서버·단일 계정·고비용 학습이라는 제약 위에서 테스트가 서로를 깨뜨리지 않게 만드는 구조였습니다. 다음 장부터 그 구조를 설명합니다.

먼저 제약을 정의했다

테스트를 짜기 전에, 이 제품이 자동화에 무엇을 허용하지 않는지부터 정리했습니다. 아키텍처의 모든 결정은 아래 다섯 가지 제약에서 나왔습니다.

제약	그래서 무엇이 어려워지는가	설계로 답한 방식
산출물의 소속 모든 노드는 자기 프로젝트 안에서만 유효	고정 프로젝트를 재사용하면 실행할 때마다 노드가 쌓여 결국 스위트가 깨진다	테스트마다 전용 프로젝트 생성 → 종료 시 삭제를 공용 헬퍼로 강제
실서버 대상 목(mock) 없이 3개 실행경	실패를 임의로 만들 수 없고, 서버 상태가 테스트 결과에 그대로 개입한다	서버 정책을 실험으로 규명해 전제로 삼고 , 장애는 요청 차단·주입으로 재현
단일 공유 계정 + SSO	동시 로그인인 앞 세션을 끊어 병렬 실행 자체가 성립하지 않는다	세션 단위가 계정이 아니라 기기(User-Agent)임을 규명 → worker별 UA 분리
고비용 작업 모델 학습 30분~3시간	소비 테스트마다 학습을 다시 돌리면 스위트를 현실적으로 운영할 수 없다	학습 결과를 픽스처에 기록해 재사용 — 타입당 1회로 고정
비동기 장기 작업 완료 신호가 일시적 UI	토스트를 기준으로 삼으면 부하가 걸릴 때 false PASS 가 난다	노드 상태 · 다운로드 파일 · 요청 페이로드를 판정 근거로 사용

화면이 아니라 데이터를 단언한다 — 예컨대 리포트 검증은 화면 표시만 보지 않고 **다운로드한 zip을 파싱해 서버 원본 값과 대조**합니다. 이미지 수 · 모델 사이즈 · Threshold가 **UI · 파일 · 생성 요청 페이로드** 세 곳에서 모두 일치하는지 확인합니다. “떠 보이는 것”과 “**실제로 된 것**”을 구분하는 것이 이 시스템의 판정 기준입니다.

4가지 설계 원칙

“모든 산출물(노드)은 만들어진 프로젝트 안에서만 쓸 수 있다”는 **제품의 도메인 제약을 아키텍처의 출발점**으로 삼았습니다.

1. 격리 — 테스트마다 전용 프로젝트를 만들고 지운다

고정 프로젝트 ID 재사용은 실행마다 노드가 쌓여 결국 깨집니다.
모든 스위트가 **시작 시 생성, 종료 시 삭제**하는 생명주기를 공용 헬퍼로 강제.

2. 비용 — 비싼 학습은 1회, 소비는 픽스처 재사용

모델 학습은 **30분~3시간**. 학습 성공 시 프로젝트 ID를 픽스처에 기록하고 프로라벨·내보내기·IoU 검증이 재사용해 **재학습 비용을 제거**.

3. 실패 격리 — 한 모델의 실패가 다른 모델을 막지 않는다

모델 타입별 Playwright project를 **동적 생성**하고 의존을 “자기 모델”로만 한정. 항상 실패하던 제품 버그(IoU)는 **의존 그래프 끝으로 분리**.

4. 사실 검증 — 화면이 아니라 데이터를 단언한다

완료 토스트 같은 일시적 UI 대신 **노드 상태 · 다운로드 파일 · 요청 페이로드**를 판정 근거로 사용. “떠 보이는 것”과 “**실제로 된 것**”을 구분.

단일 계정 · 실서버 제약 아래에서 병렬을 성립시키기

이것이 이 프로젝트의 핵심 난제였습니다.

🔑 worker별 User-Agent 분리

SSO가 “같은 UA 동시 로그인 = 같은 기기 재로그인”으로 앞 세션을 끊는 것을 **실험으로 규명**하고, worker마다 UA 토큰을 부여해 **3-worker 동시 로그인** 성립.

⚡ 파일 내 병렬

단일 직렬 전처리 스위트를 **노이즈 / 흐름 / 회전 3그룹**으로 분할, 각 그룹이 전용 프로젝트를 갖고 worker에 분산.

19.3분 → 11.8분

-39%

🏢 서버 정합

서버 **동시 작업 상한(3)**에 맞춰 worker 수와 대기 정책을 정합, 완료 대기 한도를 “**최장 학습 잔여 + 여유**”로 산정해 큐 대기 오탐 제거.

버린 것도 기록합니다 — 세션 문제 우회용으로 만들었던 탭 orchestration 구조는 **단일 실패점**이므로 **폐기**했습니다. 더 단순한 정답이 나오면 이미 만든 것도 버립니다.

스위트가 두 배 넘게 늘어나는 동안, 아키텍처는 다시 쓰지 않았다

초기 구축 이후 **모델 성능 비교 · 회귀 테스트** 영역이 더해지며 스위트 규모는 두 배 이상이 됐습니다. 이때 한 일은 구조를 고치는 것이 아니라 **기존 구조 위에 스위트를 엮는** 것이었습니다. 자동화의 진짜 비용은 처음 만들 때가 아니라 **늘려갈 때** 발생하기 때문에, 그 지점을 먼저 설계했습니다.

확장이 싸지도록 미리 심어둔 것들

설계 장치	새 스위트를 추가할 때 생기는 이점
공용 생명주기 헬퍼	프로젝트 생성·삭제를 매번 다시 짜지 않는다. 새 스위트는 격리를 공짜로 얻는다
학습 픽스처	학습 결과를 소비하는 테스트는 학습을 다시 정의할 필요가 없다 — 모델을 쓰는 새 검증일수록 추가 비용이 낮다
모델 타입별 project 동적 생성	모델 타입이 늘어도 설정 파일을 손으로 늘리지 않는다
의존 그래프로 좁힌 실패 전파	새 스위트가 실패해도 기존 스위트를 오염시키지 않는다 — 추가에 대한 심리적 저항이 사라진다
데이터 기반 판정 규약	“무엇을 근거로 통과를 선언하는가”가 이미 정해져 있어 새 TC의 판정 기준을 매번 협의하지 않는다

그래서 커버 범위는 이렇게 넓어졌다



검증의 질문이 바뀌었다 — 초기 스위트가 “기능이 도는가”를 물었다면, 확장된 스위트는 “모델이 내놓는 결과가 이전만큼 좋은가”와 “고친 것이 유지되는가”를 묻습니다. AI 제품에서는 **아무것도 실패하지 않으면서 품질만 조용히 나빠지는 일**이 가능하기 때문에, 기능 검증만으로는 릴리스를 보증할 수 없다고 판단했습니다.

이 슬라이드에서 보여주려는 것 — 커버리지 숫자가 아니라, **늘어날 것을 전제로 짜인 구조와 무엇을 검증할지에 대한 판단이 함께 성장했다는 사실**입니다.

증거로 원인을 확인한 5건

모두 **trace** · **DOM 스냅샷** · **리포트 타임라인**으로 원인을 확인한 뒤 고쳤습니다. 추측으로 고친 것은 없습니다.

CASE 1 — 로그인은 됐는데 로그인 화면으로 돌아온다

SSO 리다이렉트 race

로그인 직후 **goto** 로 끼어들면 SSO의 자동 리다이렉트 체인이 끊겨 미인증 상태가 됨을 단계별 URL 진단 스크립트로 확정.
이동을 금지하고 자동 리다이렉트 도달만 대기하도록 변경, 신/구 로그인 방식을 placeholder로 자동 분기해 **3개 환경이 한 헬퍼를 공유**.

CASE 2 — “수동은 되는데 왜 자동화만 안 되지?”

세션 정책 규명과 병렬 복구

“계정당 단일 세션”이라는 그럴듯한 가설에 머물지 않고 모순 (“수동은 된다”)을 실험으로 파고들어, 세션이 계정이 아니라 **기기 (User-Agent) 단위**임을 규명. worker별 UA 토큰 한 줄로 해결
→ cloud 병렬 **1 → 3 worker** 복구.

CASE 3 — 6분짜리 학습이 26초 만에 “완료”

false PASS

통과한 TC가 범인인 케이스. 리포트 타임라인 재구성 + DOM 스냅샷으로 “준비 구간(스피너·버튼 모두 부재)”이 병렬 부하에서 길어져 완료 판정을 오판함을 확인. 제품의 진행 토스트(“N% · 예상 종료 시각”)를 판정에 추가해 전 구간 보호.

CASE 4 — 캔버스가 하얗게 비는 글리치

제품 버그 확정 + 재현 스크립트

trace의 DOM 스냅샷 시점별 추적으로 React Flow의 인라인 **visibility:hidden** 고착을 확인, 트리거(모달 닫힘 직후 ~200ms 내 소켓 알림)까지 특정. **단독 재현 스크립트(CPU 스로틀로 부하 모사)**를 제작해 제품 버그로 리포트.

CASE 5 — 40초가 모자라서 실패한 검증

큐 대기를 타임라인으로 증명

전형적인 “재현 안 되는 flaky”를 전체 테스트의 시작/종료 시각으로 서버 작업 슬롯 점유를 재구성해 규명 — 코드 버그가 아니라 동시 작업 상한(3)을 긴 학습이 점유하는 동안의 **구조적 대기**. **flaky는 “운”이 아니라 원인이 있는 결함**으로 다뤘습니다.

🔍 공통 접근

- trace DOM 스냅샷 시점별 추적
- 리포트 타임라인 재구성
- 요청 페이로드 가로채기
- 단독 재현 스크립트 제작
- 가설과 모순되는 사실을 끝까지 파고들기

결과가 팀에 도달해야 자동화가 완성된다

리포팅 & 운영



Daisy Auto Test Result

오후 7:34

 DAISY Automation Test 결과

 테스트명 : 모델성능비교, 비
디오라벨링, regression TC 추가
검증

 소요 시간 : 1시간 30분 28초

 PASS : 502

 FLAKY : 0

 FAIL : 0

 SKIP : 0

 Total 502 ·  2026/08/17/19:29

봇이 추가한 Daisy Auto Test Result

실행이 끝나면 리포터가 결과 카드를 채널에 올린다. PASS/FAIL/FLAKY/SKIP 네 상태와 소요 시간, 검증한 스위트명이 함께 남는다.

- 커스텀 Slack 리포터(자작) — 실행 종료 시 결과 카드(PASS/FAIL/FLAKY/SKIP) + 경량 리포트 zip 자동 전송, 실패 존재 시 색상 전환
- 경량 첨부 설계 — video·trace 제외 zip만 첨부(수 GB → 1~2MB), 로컬에는 전체 아티팩트를 실행별 타임스탬프 폴더로 보존
- 대화형 실행기 — `npm run test:cloud` 한 줄로 테스트명·Slack 전송 여부를 묻고 실행. CI에서는 무질문 자동, 검증용 부분 실행이 팀 채널을 오염시키지 않도록 전송은 opt-in
- 보안 — trace에 평균 기록되는 자격 증명을 인지하고 계정 정책·gitignore·첨부 제외로 노출 경로 차단

성과 요약

항목	Before	After
릴리스 회귀 검증	수동 · 1일 이상	명령 한 줄 · 무인 실행 + Slack 자동 보고
cloud 병렬 실행	SSO 개편 후 1 worker	3 worker 복구
전처리 스위트	직렬 19.3분	그룹 병렬 11.8분 (-39%)
모델 학습 비용	소비 테스트마다 재학습	타입당 1회 · 픽스처 재사용
스위트 확장	추가할수록 구조 수정 필요	규모 2배 이상 확장 · 아키텍처 무변경
검증의 질문	기능이 도는가	+ 결과 품질이 유지되는가 · 고친 것이 유지되는가
제품 품질 기여	—	자동화가 잡은 제품 결함을 증거와 함께 리포트 (수정 확인 후 회귀 테스트로 고정)

이 프로젝트에서 보여준 방식

- 증거로 원인을 **확정한다** — trace DOM 스냅샷, 리포트 타임라인 재구성, 요청 페이로드 가로채기
- 가설은 **실험으로 깬다** — 그럴듯한 결론에 안주하지 않고 모순을 파고들어 진짜 정책을 찾는다
- 더 단순한 답이 나오면 **버린다** — 동작하던 구조도 단일 실패점이면 미련 없이 폐기
- 실패를 **설계한다** — 제품 버그는 실패하는 테스트로 고정해 추적하고, 의존 그래프로 실패 전파를 차단

Playwright

Node.js

E2E 자동화

MLOps 파이프라인 검증

병렬 설계

픽스처 전략

Slack 리포터

자동화의 마지막 병목은 “실행 권한”이었다

500여 TC 규모로 자란 E2E 자동화를 “만든 사람만 돌릴 수 있는 자산”에서 팀 공용 도구로 옮겼습니다. 설치도 명령어도 없이 사내망 브라우저 주소 하나로 실행하고 리포트까지 회수하는 셀프서비스 러너를, 설계 · 구현 · 사용자 문서화까지 단독으로 진행했습니다.

0 단계

설치

0 줄

명령어

자동

선행 의존 해결

복원

실행 세션 재접속

원클릭

리포트·스크린샷 회수

자동화의 가치는 커버리지가 아니라 **실행 빈도**에서 나옵니다. 아무리 잘 만들어도 실행이 한 사람을 거쳐야 하면 **그 사람의 일정이 곧 검증 주기의 상한**이 됩니다. 러너는 그 상한을 없애기 위해 만들었고, 해결해야 할 문제는 세 가지였습니다.



실행 병목

검증이 필요할 때마다 QA 담당자를 거쳐야 하고, 담당자가 자리에 없으면 검증도 멈춤



공유 계정 충돌

테스트가 실서버 계정 하나를 함께 쓰기 때문에, 두 사람이 동시에 돌리면 서로가 만든 프로젝트를 지워 **둘 다 실패**



결과 회수 부재

서버에서 돌린 실행의 리포트·스크린샷을 개인 PC로 가져올 경로가 없음



설계 목표 — “터미널을 몰라도 돌릴 수 있다”가 아니라, “도구를 모르는 사람이 눌러도 잘못될 수 없다”. 셀프서비스는 권한을 여는 일이 아니라, **열어도 안전한 상태를 만드는 일**이라고 봤습니다.

실행 전 — 누르기 전에 잘못될 수 없게

화면에 드러난 결정들과, 그렇게 정한 이유입니다. 아래 화면은 실제 운영 중인 러너입니다.

판단 1 막되, 막는 이유를 함께 준다

공유 계정이라는 제약은 없앨 수 없으므로 러너가 동시 실행을 차단합니다. 문제는 차단 자체가 아니라 **차단이 “고장”으로 보인다는 점**이었습니다. 그래서 막을 때 **누가·무엇을·몇 분째** 돌리는 중인지를 같은 화면에 노출했습니다.



실행이 막힐 때, 막는 데서 끝나지 않고 누가·무엇을·몇 분째 돌리는지 함께 노출한다.

왜 이렇게

사용자가 기다릴지 물어볼지를 스스로 판단할 수 있으면, 차단은 장애가 아니라 정보가 됩니다. 도구에 대한 신뢰는 **되는 기능이 아니라 안 될 때의 설명**에서 갈린다고 봤습니다.

판단 2 선행 의존은 사용자가 아니라 러너가 안다

프로라벨을 검증하려면 모델이 먼저 학습돼 있어야 합니다. 이 의존 관계를 **사용자가 알아야 한다면 셀프서비스가 아니므로**, 러너가 선행 테스트를 자동으로 함께 선택하고 **🔒**로 표시합니다. 마우스를 올리면 무엇 때문에 포함됐는지까지 나옵니다. 실제로 **1건을 체크하면 16건이** 선택됩니다.



프로라벨 1건을 체크하면 16건이 선택된다. **🔒** 15건은 러너가 자동으로 채운 선행 테스트다.

왜 이렇게

자동으로 채우기만 하면 “왜 16건이지?”라는 불신이 생깁니다. **자동화되되 근거를 열어두는** 쪽을 택했습니다.

판단 3 예상 시간은 실제 실행 모델을 반영한다

선택 건수와 예상 소요 시간을 즉시 계산해 보여주되, 단순 합산이 아니라 **3-worker 병렬 실행이라는 실제 실행 방식을 반영한 값**을 씁니다. 환경과 언어를 바꾸면 대상 테스트케이스 목록 자체가 함께 갱신됩니다.



Daisy Auto Test Result 웹 오늘 오전 9:52

서버테스트 0828

- 요청: 김상욱
- 환경 / 언어: CLOUD / 국문
- 테스트케이스 502건 · 예상 약 1시간 33분
- [진행 상황 보기](#)

1개의 댓글



Daisy Auto Test Result 웹 오늘 오전 11:17

✓ 서버테스트 0828 — 완료

- 완료 502건
- 소요 시간: 85분 34초
- [결과 리포트 열기](#)

실행을 걸면 요청자·환경·규모·예상 시간이 채널에 공지되고, 끝나면 같은 스레드에 실제 소요 시간이 달린다.

왜 이렇게

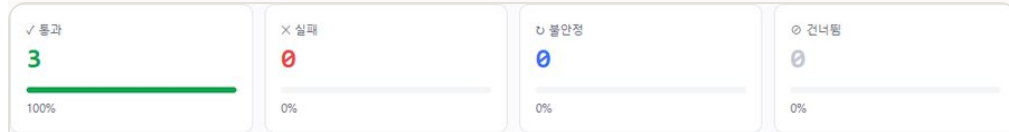
어긋난 예상치는 한 번만 틀려도 다음부터 아무도 보지 않습니다. 보여줄 수 없으면 안 보여주는 게 낫고, 보여줄 거면 맞아야 합니다.

위 실행에서 **예상 1시간 33분 → 실제 1시간 25분 34초**. 3-worker 병렬을 반영해 계산했기 때문에 나온 오차입니다.

실행 중.후 — 자리를 떠도 되고, 결과는 내 손에 남게

통과한 것도 “무엇을 봤는지”를 남긴다

자동화의 결과물이 PASS/FAIL 숫자로만 끝나면, 사람은 그 숫자를 검증할 방법이 없습니다.



리포트 상단 요약. 네 상태의 합이 전체 건수와 일치한다.

전체				
✓ 통과 ✗ 실패 ⚠ 불안정 🕒 건너뛴				
상태	테스트케이스 ID	이름	카테고리	소요
✓ 완료	common/PROJECT-TC1	프로젝트 생성	프로젝트	3초
✓ 완료	common/PROJECT-TC2	프로젝트 이름 변경 ▼ 검증 포인트 2개 1. ✓ TC1에서 생성된 projectid 가 필요합니다 확인 2. ✓ 변경된 프로젝트명 반영 확인	프로젝트	9초
✓ 완료	common/PROJECT-TC3	프로젝트 삭제 ▼ 검증 포인트 2개 1. ✓ TC1에서 생성된 projectid 가 필요합니다 확인 2. ✓ 프로젝트 카드 삭제됨 확인	프로젝트	4초

검증 포인트를 펼친 모습. 줄 끝의 “화면”을 누르면 그 시점 스크린샷이 열린다.

각 테스트의 **검증 포인트**를 펼치면 무엇을 어떤 순서로 확인했는지가 나오고, 줄 끝의 “화면”을 누르면 그 시점 스크린샷이 열립니다. **통과한 테스트에도 동일하게 붙습니다.**

실패 항목은 **그 시점 화면이 리포트 파일 안에 들어 있습니다**. 링크나 별도 첨부 없이 리포트 하나만 전달해도 상대가 그대로 볼 수 있습니다 — 개발자에게 재현 요청을 넘길 때의 **왕복을 줄이기 위한 설계**입니다.

상태를 네 가지로 분리한 이유

표시	뜻	취급
완료	검증을 통과	정상
실패	기대 결과와 다름 — 사유와 화면이 함께 제공	원인 확인
불안정	재시도해서 통과	초록이 아님 · 확인 필요
건너뛴	선행 실패 또는 중단으로 미실행	선행 결과부터 확인

왜 이렇게

“재시도해서 통과했으니 통과”로 넘기면 **flaky가 영원히 쌓입니다**. 불안정을 별도 상태로 분리해, flaky를 운이 아니라 원인 있는 결함으로 다루는 기준을 **도구 차원에서 고정**했습니다.

도구와 함께 만든 사용자 문서, 그리고 결과

04 배포만으로는 아무도 쓰지 않는다

화면 순서를 그대로 따라가는 가이드를 **도구와 함께** 작성했습니다.

- **사용자의 언어로 쓴다** — “헤드리스”, “동시성 락” 대신 “창 띄우기”, “지금 다른 실행이 진행 중입니다”
- **막히는 화면을 미리 설명한다** — 실행이 차단되는 화면을 스크린샷과 함께 보여주고 “이건 고장이 아닙니다”라고 먼저 말해 문의 자체를 줄임
- **문의가 들어올 지점을 먼저 답한다** — 실패가 많이 날 때 / 예상보다 오래 걸릴 때 / 리포트가 안 열릴 때를 트러블슈팅 섹션으로 선제 정리
- **문서를 화면에 중속시킨다** — 실제 화면 캡처를 그대로 담고, 화면이 바뀌면 문서도 함께 갱신하는 것을 전제로 구성

이 문서 자체가 산출물입니다. 도구를 만든 사람이 문서까지 책임지지 않으면 결국 문의가 다시 그 사람에게 돌아오고, 병목은 형태만 바뀌 유지됩니다.

05 결과

항목	Before	After
실행 주체	터미널 접근 가능한 QA 1인	개발·QA 팀원 누구나 (브라우저 주소 하나)
실행 준비	클론 · 의존성 설치 · 환경 변수 · 명령어 숙지	설치 0단계 · 명령어 0줄
동시 실행 사고	공유 계정 충돌로 양쪽 모두 실패	단독 실행 보장 + 실행자·경과 시간 노출
선행 조건	사용자가 의존 관계를 알고 직접 선택	러너가 자동 선택 · 포함 사유 표시
실행 중 이탈	세션 끊기면 진행 상황 유실	재접속 시 로그·진행률 복원 (타인 실행 포함)
결과 회수	개인 PC로 가져올 경로 없음	리포트 · 스크린샷 zip 원클릭 다운로드

이 작업에서 보여준 방식

- **자동화는 돌아가는 순간이 아니라, 남이 쓸 수 있을 때 끝난다** — 실행 UI와 사용자 문서를 자동화의 일부로 봄
- **막을 땀 이유를 함께 준다** — 없앨 수 없는 제약이라면 최소한 판단할 정보를 준다
- **도메인 지식이 필요한 부분은 도구가 대신 계산한다** — 선행 의존, 병렬 소요 시간
- **기준을 문서가 아니라 도구에 고정한다** — 불안정 상태 분리로 flaky 판정 기준을 강제

테스트 자동화 셀프서비스

사내 도구 개발

실행 오케스트레이션

동시성 제어

의존성 자동 해결

실시간 로그 스트리밍

QA 이네이블먼트

사용자 문서화

Playwright UI Automation · Selenium 반응형 도구

Playwright UI Automation Test

공간정보진흥원 LMS · 회원가입 폼 검증 자동화

수동으로 수행하던 회원가입 폼의 유효성 검증 테스트를 자동화한 프로젝트입니다. 기존 Selenium 대비 안정적이고 빠른 Playwright를 도입해 웹 접근성 검증과 회귀 테스트 효율성을 향상시켰습니다.

| 주요 기능

- **포괄적 폼 검증** — 아이디·비밀번호·이메일·이름·전화번호 필드 유효성 시나리오 (TC-070 ~ TC-131)
- **데이터 기반 테스트** — JSON 파일로 테스트 데이터 관리
- **Page Object 패턴** — `FormValidationPage.js` 로 선택자·메서드 분리
- **보안 테스트 포함** — SQL Injection, XSS 등 취약점 검증 데이터
- **자동 리포팅** — 실패 시 스크린샷·비디오 녹화, HTML 리포트 생성

| 성과

- **테스트 시간 80% 이상 단축**
- 코드 변경 시마다 자동 전체 검증 → 회귀 자동화
- 실제 LMS 프로젝트에 적용, 다른 웹 앱에도 이식 가능한 모듈화 설계

github.com/kx2471/playwright_Automation_Test ↗

Selenium 기반 반응형 웹 UI 테스트 자동화 도구

Python · Selenium · Tkinter GUI

다양한 화면 크기(Breakpoint)에 따른 웹 UI를 자동 테스트하고 스크린샷을 캡처하는 GUI 애플리케이션입니다. **GitHub Releases**로 **실행 파일을 배포**해 비개발자도 사용할 수 있게 했습니다.

| 주요 기능

- **GUI 기반 인터페이스** — ID/PW, URL 목록, 저장 경로를 손쉽게 입력·관리
- **자동 로그인** — 인증이 필요한 페이지의 UI 테스트 수행
- **사용자 정의 Breakpoint** — XL·LG·MD·SM 외 직접 너비 설정 (입력값 유효성 검사 포함)
- **다중 URL 일괄 테스트** — `url.txt` 목록을 순차 방문하며 전 페이지 캡처
- **멀티 브라우저** — Chrome / Edge 크로스 브라우징

| 기대효과

- 반복적인 UI 테스트·스크린샷 캡처 자동화로 개발·QA 생산성 향상
- 다양한 해상도의 UI 오류를 조기 발견해 시각적 품질과 UX 일관성 유지

03

CHAPTER

AI 활용 & 실무 도구 자체 개발

같은 업무를 두 번 반복하지 않기 위해, **필요한 도구는 직접 만드는 습관**을 일관되게 유지해왔습니다. AI를 단순히 사용하는 단계를 넘어, QA 업무 구조 자체에 어떻게 녹여 넣을지를 고민하고 **실제 도구로 만드는 단계**에 있습니다.

Regression TC Creator (Claude Agent)

MCP 도구 조합 · 3단계 게이팅

AI를 워크플로우 구성원으로

내가 설계한 QA 정책을, 그 정책을 따르는 도구로 다시 만들었습니다

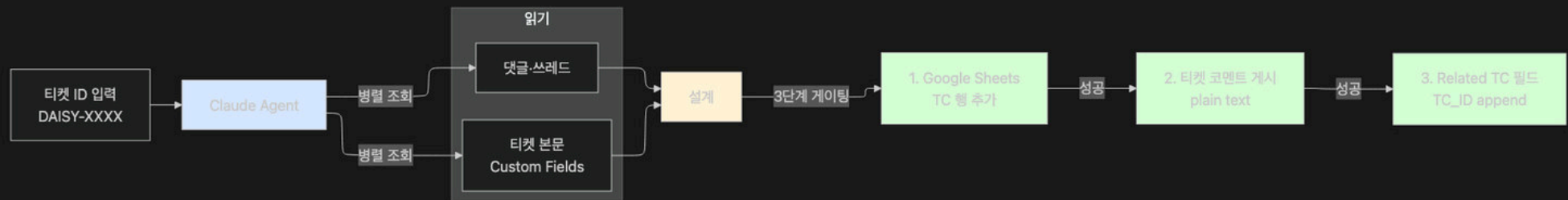
ClickUp 티켓을 분석해 회귀 테스트 케이스를 자동 설계하고, Google Sheets에 기록하고, 티켓에 코멘트로 알림과 관련 TC 필드까지 연결하는 Claude 기반 AI Agent입니다.

왜 만들었는가

QA 워크플로우를 설계하면서 **버그 티켓의 Root Cause · Corrective Action**을 기록하도록 의무화했습니다. 그런데 이 정보는 기록만 되고 활용이 안 되는 자산이 될 위험이 있었습니다 — 매 스프린트마다 여러 버그 티켓을 일일이 읽고, 회귀 TC를 손으로 설계하고, 시트에 기록하고, 원본 티켓에 연결하는 과정이 **단순 반복의 연속**이었습니다.

그래서 이 반복 작업을 **Claude Agent**로 직접 구현했습니다. 티켓 ID 하나만 주면 아래 세 가지가 자동으로 일어납니다.

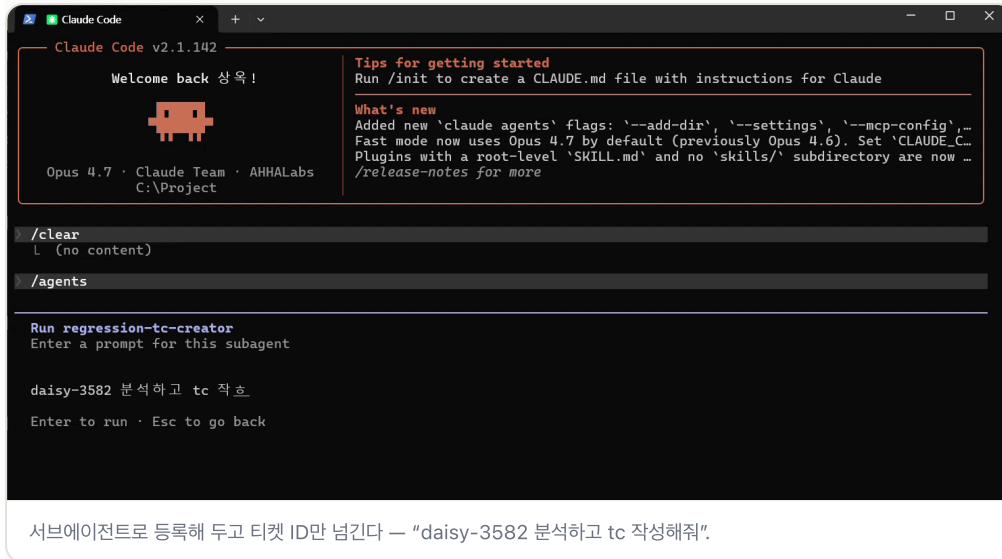
동작 흐름 — 3단계 게이팅



설계 단계에서 그린 실제 동작 흐름. 티켓 본문·Custom Fields와 댓글·쓰레드를 병렬 조회한 뒤 설계하고, 세 개의 쓰기를 순차 게이팅으로 묶었다.

세 가지 쓰기 작업은 **앞 단계가 성공해야 다음이 진행**됩니다. 일부만 게시되는 **불완전 상태**를 **원천적으로 차단**하기 위함입니다.

호출은 한 줄이면 끝난다



본 문서는 실제 업무 경험을 바탕으로 재구성·가공되었으며, 회사 기밀 보호를 위해 모든 민감 정보는 익명화 예시 패턴으로 대체되었습니다.

에이전트를 **서브에이전트로** 등록해 두고, 티켓 ID 하나만 자연어로 넘기면 됩니다. 별도 설정이나 옵션 플래그 없이 **기본 시트에 기록되도록** 기본값을 잡아, 쓰는 사람이 매번 결정할 것을 없앴습니다.

왜 이렇게

도구의 채택률은 기능 수가 아니라 **처음 쓸 때의 마찰**에서 갑니다. "무엇을 설정해야 하지?"가 한 번이라도 떠오르면 사람들은 그냥 손으로 합니다.

설계 원칙 — QA 엔지니어의 사고 과정을 명문화

1. RCA 기반 TC 설계

단순히 티켓을 요약하는 게 아니라, **과거 실패 모드를 재현하는 일**을 합니다.

- Root Cause가 명시된 경우 → 그 실패 모드를 **재현하는 명시적 케이스** 설계
- Corrective Action이 실제로 막거나 처리하는지 확인하는 **회귀 케이스** 추가
- 같은 근본 원인의 영향을 받을 수 있는 인접 경계 조건도 함께 설계

3. 양보다 질 — 억지로 채우지 않기

에이전트가 형식적으로 TC를 찍어내는 것을 막기 위해 출력량 규칙을 명확히 정의했습니다.

- **기본 목표**: 티켓당 3개 (happy-path 1 + 핵심 회귀 1 + 인접/예외 1)
- **축소**: 단순 버그(텍스트 수정, 좁은 정규식 보정)는 1~2개로 마무리 가능
- **확장**: Corrective Action이 여러 기능 영역에 걸쳐거나, Frequency가 Always이고 부하 케이스가 본질일 때
- **금지**: 형식적 happy-path나 의미 없는 변형으로 빈자리 채우기

시트에 넘기기 애매한 보조 검증 아이디어는 시트가 아닌 **코멘트의 “추가로 검증되어야 할 점”** 블록에 담아 시트와 코멘트의 역할을 분리했습니다.

2. 5축으로 시나리오 스캔

축	명세 / 예시
Happy-path	의도대로 동작하는 주요 end-to-end 시나리오
입력/데이터 변형	경계값, 일반값, 의미 있는 조합
에러/실패 조건	잘못된 입력, 누락 데이터, 타임아웃, 권한, 연동 실패
연동/외부 시스템	다른 서비스, 모델 추론, 스토리지, 백엔드 응답 기반 UI 갱신
Frequency/반복	빈번 실행, 배치/스케줄, 피크 타임 시 안정성

안전장치 — 근거 없이 쓰지 않는 구조

- **미리보기 모드** — “초안만 보여줘”라고 하면 시트·코멘트·필드 갱신을 모두 생략하고 화면에만 출력
- **부분 거부 수용** — “시트에 쓰지 마”, “코멘트는 게시하지 마” 같은 지시를 개별 처리, 거부된 단계 이후는 자동으로 함께 생략
- **fallback 시트 탐색** — 기본 시트 접근 불가 시에만 search로 후보 1~3개 제시, 확정 안 되면 **쓰기 전체 보류**
- **필드 있을 때만 갱신** — Related TC 필드가 없으면 생성하지 않고 건너뛰기
- **실패 알림 레이어** — 각 단계 성공 여부를 메타 라인으로 명시

핵심 쓰기 구조 — 3단계 게이팅

Step 1. Google Sheets 기록 (정본)

14컬럼 양식으로 시트 마지막 행 다음에 직접 기록합니다.

컬럼	값
TC_ID	마지막 TC_ID + 1 순차
TC명 / 사전조건 / 절차 / 기대결과	Claude가 설계
레이블	Regression Test (고정)
작성자	Claude (고정)
REQ Traceability	출처 티켓 custom_id
타입·기능영역·세부영역·모델타입	공란 (QA가 분류)
P/F	공란 (QA가 실행 후 채움)
비고	회귀 근거, 데이터 가정, 재

Agent가 추가한 행은 **주황색 배경**으로 칠해 사람이 작성한 TC와 즉시 구분되도록 했고, `wrapStrategy: WRAP` 을 적용해 셀 안 줄바꿈이 실제로 보이도록 했습니다.

Step 2. 티켓 코멘트 게시 (알림)

TC 표는 코멘트에 넣지 않고, **TC는 시트가 정본**이라는 원칙을 지킵니다. 코멘트에는 아래만 들어갑니다.

- **[간단 분석]** — 핵심 증상, 환경, Root Cause/Corrective Action을 2~3문장으로
- **[추가로 검증되어야 할 점]** — 시트에 안 들어간 보조 검증 아이디어 3~5개
- **[관련 예외/엣지 케이스]** — 파생 변형 2~4개
- **[시트 추가 위치]** — 추가된 TC_ID 범위 + 시트 링크

ClickUp 코멘트는 마크다운을 렌더링하지 않아 ****굵게**** · **# 헤더** · **- 불릿** 이 모두 raw 문자로 노출됩니다. 그래서 제목은 대괄호 **[...]** , 불릿은 가운데점 **·** , 강조는 **ALL CAPS: 패턴**으로 **시각 구조를 잡는 규칙을 따로 설계**했습니다.

Step 3. Related TC append (추적성)

생성한 TC_ID를 티켓의 Related TC 커스텀 필드에 **기존값 보존한 채 append**합니다.

이 과정으로 **“이 버그를 계기로 생성된 회귀 TC”**의 양방향 추적성이 자동으로 남게 됩니다.

이는 제가 설계한 QA 워크플로우의 세 번째 원칙(**Root Cause 의무화**)에서 끝나지 않고, 그 기록이 검증 자산으로 이어지는 고리를 만드는 단계입니다.

쓰기 후 검증

쓰기 직후 같은 범위를 다시 읽어 **실제 들어간 행 번호·TC_ID 범위를** 검증하도록 했습니다. 드물게 행 충돌이 나면 사용자에게 바로 알립니다.

실제 결과

DAISY TC_신규

파일 수정 보기 삽입 서식 데이터 도구 확장 프로그램 도움말

메뉴

100%

W %

123

기본값 ...

11

B I

A

TC_ID	타입	기능영역	세부영역	모델타입	TC명	사전조건	테스트 절차	기대결과	레이블
TC_871					장바구니 - 동일 1회용 쿠폰 중복 적용 차단 및 페이지 재진입 시 적용 상태 동기화	- 일반 회원으로 로그인된 상태 - 장바구니에 상품 1개 이상 담겨 있음 - 사용자 쿠폰함에 1회용 "5,000원 할인 쿠폰" 1매가 발급되어 있음	1) 장바구니 페이지 진입 2) 쿠폰 적용 버튼 클릭하여 쿠폰 선택 모달 열기 3) "5,000원 할인 쿠폰" 선택 후 적용 버튼 클릭 4) 결제로 진행하지 않고 카테고리 또는 홈으로 이동 5) 다시 장바구니 페이지로 복귀하여 쿠폰 적용 상태 확인 6) 쿠폰 적용 버튼을 다시 클릭하여 쿠폰 선택 모달 재진입 7) 이미 적용된 "5,000원 할인 쿠폰"의 상태 확인 및 재선택 시도	- 5단계: 장바구니 복귀 시 "5,000원 할인 쿠폰"이 이미 적용된 상태로 유지되고 주문 요약에 "할인 5,000원"으로 표시됨 - 6단계: 쿠폰 선택 모달에서 해당 쿠폰이 "사용 중" 상태로 표시되고 재선택이 차단됨 - 7단계: 강제로 재선택을 시도하면 토스트 메시지 "이미 적용된 쿠폰입니다."가 노출되고 중복 적용되지 않음 - 주문 요약 총 할인 금액이 -10,000원이 아닌 -5,000원으로 유지됨	Regression

Step 1 — 마지막 행 다음에 회귀 TC가 추가된다. Agent가 넣은 행은 주황색 배경이라 사람이 쓴 TC와 즉시 구분된다. (TC_871 외 실제 TC는 비식별 처리)

구현 과정에서 마주친 실

만들면서 부딪힌 구체적인 이슈들과, 그걸 어떻게 해결했는지입니다.



Sheets API

add_rows

가 잘못된 위치에 행을 넣는 문제

처음에는 `add_rows` (내부적으로 `values.append` 호출)를 쓰는 게 자연스러워 보였습니다. 그러나 시트에 드롭다운-조건부 서식이 걸린 빈 영역이 있으면 이 API가 그 영역까지를 "테이블"로 인식해 **마지막 데이터 행에서 10~20행 떨어진 곳에 행을 박아버리는 문제**가 있었습니다.

→ 해결: `add_rows` 를 포기하고, 마지막 데이터 행 번호 + 1부터 시작하는 **명시적 셀 범위**를 계산해 `update_cells` 로 직접 쓰도록 설계 변경.



ClickUp 코멘트가 마크다운을 렌더링하지 않는 문제

마크다운으로 깔끔하게 쓰면 화면에서 `**강조**` · `##` 헤더 · `-` 불릿 이 모두 글자 그대로 노출되었습니다.

→ 해결: 마크다운 전면 금지로 규칙을 바꿔, 제목은 대괄호 `[...]` , 불릿은 `·` (가운데점), 강조는 `ALL CAPS:` 패턴으로 **plain text**에서도 시각 구조가 남도록 재설계.



TC_ID 체계는 이미 존재했으므로 "추측"하지 않음

시트 마지막 행의 TC_ID 값을 **먼저 읽고**, 그 형식을 유지한 채 일련번호만 증가시키도록 했습니다. 순수 숫자면 +1, 접두어가 있으면 접두어 유지.

Agent가 출처·형식을 마음대로 만들지 않도록 제약하는 것이 핵심이었습니다.

Step 2.3 - 티켓에 붙서 코멘트가 게시되고(오른쪽), Related TC 필드에 TC_871이 연결된다(왼쪽 하단). Agent가 읽는 입력인 Root Cause · Corrective Action 필드도 같은 화면에 보인다.



이 작업이 보여주는 것

- **정책 설계자와 도구 빌더가 같은 사람**이라는 장점 — RCA 의무화 정책을 만든 사람이, 그 정책이 남긴 데이터를 활용하는 도구를 직접 만든 일관성
- **AI를 도구가 아닌 워크플로우 구성원으로 설계** — 단순 프롬프트 래퍼가 아니라 MCP 도구 조합 · 게이팅 · 안전장치까지 포함한 시스템
- **QA 엔지니어의 사고 과정 자체를 명문화** — 5축 스캔, RCA 기반 설계, 양보다 질 원칙
- **실제 구현 디테일이** 앞서는 경험 — API 버그 우회, 렌더링 제약 대응, 검증 단계 설계

Claude Agent

MCP (ClickUp · Google Sheets)

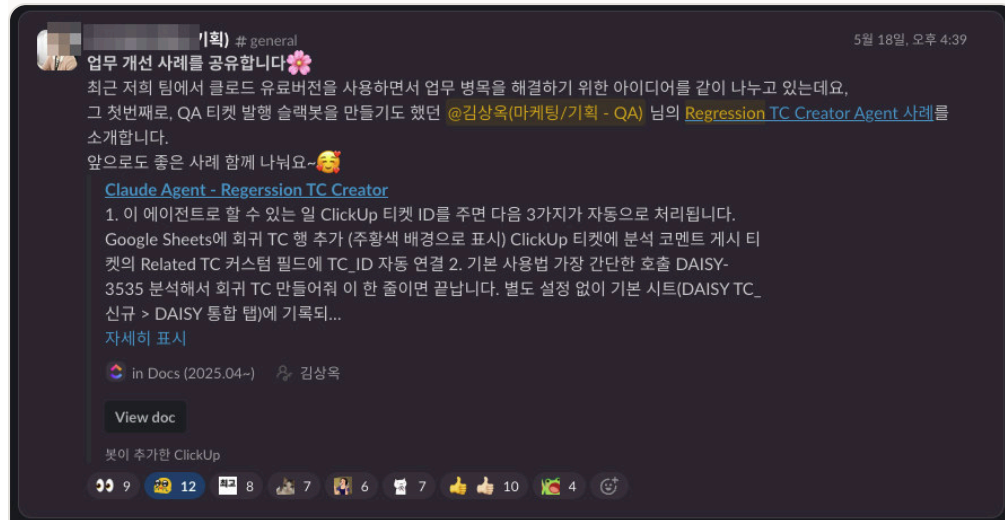
LLM 워크플로우

3단계 게이팅

추적성 자동화

내 도구에서 팀의 사례로

만들어 쓰는 것과, 조직이 받아들이는 것은 다른 문제입니다. 이 에이전트는 제가 홍보한 것이 아니라 다른 팀이 사내 개선 사례로 소개하면서 알려졌습니다.



사내 #general 채널에 “업무 개선 사례”로 공유된 Regression TC Creator. 60개 이상의 리액션이 달렸다. (공유자 정보는 비식별 처리)

확산을 염두에 두고 만든 것들

- **한 줄 호출** — DAISY-3535 분석해서 회귀 TC 만들어줘 . 설정·플래그 없이 기본 시트에 기록되도록 기본값을 잡음
- **결과가 눈에 보이게** — 주황색 행, 티켓 코멘트, 필드 연결. “돌렸는데 뭐가 달라졌는지 모르겠다”가 없도록
- **사용 가이드 동봉** — 할 수 있는 일 · 기본 사용법 · 옵션을 사내 문서로 함께 배포

이 반응이 의미하는 것

- 도구가 **QA 팀 밖에서도 이해 가능한 형태**였다는 것
- AI 활용이 개인 실험이 아니라 **팀이 따라 할 수 있는 사례**가 됐다는 것
- 같은 맥락에서 **QA Request** 봇도 함께 언급됨 — 단발성이 아니라 반복적으로 도구를 만들어 온 사람으로 인식됨

돌아보며

좋은 도구를 만드는 것보다 **남이 쓰게 만드는 것이 훨씬 어렵다**는 걸 테스트 러너와 이 에이전트에서 반복해서 배웠습니다. 그래서 이제는 도구를 만들 때 **사용 가이드와 첫 실행 경험을 같은 작업으로 취급**합니다.

04

CHAPTER

경력 & 프로젝트

3개사 · 10개 프로젝트. 자동차 임베디드에서 시작해 공공·교육 웹을 거쳐 산업 AI 비전 플랫폼까지, **도메인은 달라져도 늘 같은 방식으로 일해왔습니다.**

아하랩스 — Daisy 플랫폼 · 고객사 도입 검증

소디스 — LMS · 자율주행 · CCU

와이드넷 — TMAP AUTO for Benz

경력 사항

2026.02 - 현재

아하랩스

QA Engineer · SW QA 담당

산업 AI 비전 검수 플랫폼 Daisy의 품질 검증 총괄. QA 프로세스 0 → 1 구축, MLOps E2E 회귀 자동화 시스템 1인 구축·운영, 다수 고객사 배포 QA 동시 운영.

2025.06 - 2026.01

소디스

기술연구소 주임연구원 · SW QA 담당

QA 조직이 부재한 환경에 애자일 스크럼 · Git 브랜치 전략 · Jenkins CI/CD를 직접 수립. 공간정보진흥원 LMS에 Playwright 자동화 도입, 대전 자율주행 상용화지구 조성사업 및 커넥티드카 국책과제 검증 수행.

2023.02 - 2025.06

와이드넷 엔지니어링

미러링검증1팀 주임 · 모빌리티 검증 테스트 엔지니어

TMAP AUTO for Mercedes-Benz 내비게이션 시스템 검증. ASPICE Level 1 SWE.6 인증 획득 기여, 2주 스프린트 기준 700건 이상 TC 작성·검증, HILS 기반 조기 검증 환경 구축.

세 회사를 관통하는 한 가지 방식

도메인은 자동차 임베디드 → 공공·교육 웹 → 산업 AI 비전으로 바뀌었지만, 들어가서 한 일의 순서는 늘 같았습니다.

- **체계를 세운다** — 표준이 있으면 정착시키고(ASPICE), 없으면 만든다(워크플로우·배포 게이트)
- **반복을 자동화한다** — 한 번 손으로 한 검증은 다시 손으로 하지 않는다
- **필요한 도구를 직접 만든다** — 프로세스로 안 풀리면 코드로 푼다

각 회사에서 남긴 것

- **와이드넷** — 정형화된 표준을 지키며 대규모 검증을 굴려본 경험 (ASPICE Level 1 SWE.6 기여)
- **소디스** — 표준이 없는 곳에 프로세스와 CI/CD를 세워본 경험, 첫 자동화 도입
- **아하랩스** — 프로세스와 자동화를 둘 다 0에서 설계하고, AI를 워크플로우에 편입시킨 경험

Daisy 플랫폼과 고객사 도입 검증 2026.02 ~ 현재

플랫폼 자체의 품질과, 그 플랫폼이 각 현장에 들어갈 때의 품질을 함께 봅니다. 도메인은 제조 · 유통 · 에너지 · 통신으로 갈리지만 검증 대상은

모두 AI 판정 결과입니다.



Daisy AI Ops Platform

2026.02 ~ 현재 · QA Engineer

스마트팩토리 AI 비전 검사 플랫폼. 기능 검증 · 배포 QA와 함께 **QA 프로세스를 0에서 구축**했습니다.

- 워크플로우 · 배포 게이트 · RC 검증 · RCA 체계 설계 Ch.01
- MLOps 무인 회귀 검증 시스템 · 셀프서비스 러너 Ch.02
- Claude Agent 기반 회귀 TC 자동 생성 Ch.03

요구사항 ⇄ TC ⇄ 결함 추적성 체계를 구축해 신규 기능 **요구사항 커버리지 100%**를 달성했고, 월간 품질현황보고를 담당합니다.



대한송유관공사 — AI 누유 검출

2026.04 ~ · 검증 담당 · 호서 구간 시범

배관 감시 시스템의 **오알람 저감**을 위한 AI 누유 예측 모델과 대시보드 검증.

- **지표를 어떻게 세는지부터 확인** — 누유감지율·오알람률의 **1분 단위 카운팅 집계 기준**을 검증. 모델이 맞았는지보다 맞았다고 판정하는 방식이 먼저
- 압력 **초당 1,000건** 규모 시계열 처리와 TCP 소켓 준실시간 수집 연동 검증
- 통신 단절·재연결 구간의 데이터 유실 시나리오와 캐시 보존 기간 제약을 반영한 장애 대응 조건 검증



부천 — 필터 원단 불량 검출

2026.05 ~ 2026.07 · 검증 담당 · 3개 라인

생산 라인에 광학계와 AI PC를 설치해 **불량을 실시간 검출하고** 검단기를 정지시키는 솔루션 도입 검증.

- **설비가 실제로 멈추는지까지가 검증 범위** — 촬영 → 이상 감지 → 검단기 정지 트리거 전송 → 수신 동작 → 검출 내역 확인 → 재구동을 E2E로
- AI PC 1대 : 검단기 1대 구성에서 **3개 라인 동시 운영 시 실시간 처리** 검증
- 조명·광학계 고정 상태가 흔들릴 때의 검출 편차 확인 — 현장 환경이 곧 변수



기아자동차 — 토크 검사 ABS

2026.06 ~ 2026.08 · 검증 담당 · PoC

토크 스페너 측정값이 **차량의 어느 체결부에서 측정된 것인지**를 영상 딥러닝으로 판별하는 과제의 타당성 검증.

- **목적에 맞는 판정 기준을 세웠습니다** — 완전 판별이 아니라 **실행 가능성 확인**이 목적인 PoC라, 통과/실패가 아닌 가능 여부의 근거를 산출물로
- 촬영 각도 · 조명 · 거리를 달리한 영상 세트로 판별 가능 여부 검증
- 매칭 정확도 측정 후 **오판 케이스를 분류**, 작업 순서 오류 시나리오로 **기존 순서 기반 저장 방식의 한계**를 드러냄

LG U+ — KLK 온톨로지 대시보드

2026.06 ~ · 검증 담당

데이터 · 메타데이터 · 비즈니스 용어를 **AI 온톨로지**로 구조화해 관리·운영하는 대시보드 검증.

- **한 곳을 바꿨을 때 연결된 것이 따라오는지**
— 용어 등록·수정·삭제 시 연결된 온톨로지 반영 **정합성** 검증
- 데이터 · 메타데이터 · 비즈니스 용어 간 **관계 매핑 결과** 검증
- 조회·검색·필터 기능 검증 및 결함 리포트

뉴퍼마켓 — LLM 기반 ERP 매칭

2026.02 ~ 2026.04 · QA Engineer

LLM으로 ERP 품목을 자동 매칭하는 시스템의 기능 검증과 **응답 정확도 검증**.

- **LLM 매칭 결과의 정확도 검증 기준을 수립**
— 정답이 하나로 정해지지 않는 출력을 판정 가능한 형태로
- Figma Export 기반 UI 요구사항 분석 → TC 설계
- HTTP 통신 보안 취약점 조사 → 개선 사항 도출

자율주행·커넥티드카·공공 웹·자동차 내비게이션 2023.02 ~ 2026.01

커넥티드카 CCU 국책과제

2025.08 ~ 2026.01 · QA Engineer · 소디스

차량 텔레매틱스 CCU의 원격제어 · 데이터 송수신 검증과 소프트웨어 신뢰성 검사.

- KOLAS 공인 소프트웨어 신뢰성 검사 통과
- CANoe 기반 HILs 장비 네트워크 송수신 검증, DCM 연동 CAN 통신 검증
- Linux 기반 보안 테스트 · 사이버보안 검증 절차서/결과서 작성

대전 자율주행 상용화지구

2025.06 ~ 2026.01 · QA Engineer · ETRI 주관

자율주행 버스 애플리케이션의 기능 검증 및 보안 취약점 분석.

- 실차 검증에 직접 참여 — 현장에서 품질 이슈 대응 및 개선
- CAN 통신 · ETRI 서버 프로토콜 기반 송수신 검증
- Jira 기반 결함/테스트 이슈 관리 체계 구축 및 운영

공간정보진흥원 LMS

2025.06 ~ 2026.01 · QA · 소디스

교육포털 학습관리시스템 개발·유지보수. **자동화를 처음 도입한 프로젝트**입니다.

- Playwright 기반 UI 자동화 도입 상세 · Jenkins CI/CD 구축
- Xray 도입으로 TC-결함 추적성 확보 상세
- KRDS 웹 접근성 반영 UI 요구사항 정의서 작성, REST API 검증 상세

TMAP AUTO for Mercedes-Benz

2023.03 ~ 2025.06 · Test Engineer · 와이드넷

Benz 차량 탑재 내비게이션 검증. **정형화된 표준 아래에서 대규모 검증을 굴러본 경험.**

- ASPICE Level 1 SWE.6 인증 획득에 기여
- 2주 스프린트 기준 700건 이상 TC 작성 및 검증
- HILS 테스트 환경 구축 — 실차 대비 조기 검증 구조 마련
- 비기능 KPI 측정 스크립트 개발로 검증 리소스 30% 절감 도구

05

CHAPTER

QA 산출물 · API 검증 · 업무 도구 · 기술 스택

실무에서 작성한 표준 산출물과, 반복 업무를 줄이기 위해 직접 개발한 도구들입니다. 각 문서는 회사 보안 정책에 따라 요약본 형태로 정리되어 있으며, 예시는 실제 프로젝트 구조와 유사하게 재구성한 **가공된 가상 데이터**입니다.

SW Requirement · Test Case · Defect Ticket

Xray · Confluence

API · 보안 검증

업무 자동화 도구 6종

기술 스택

SW Requirement 요구사항에서 TC로

Functional SW 리뷰에 정기적으로 참여하여 테스트 엔지니어의 관점에서 조언하고 진행 방향을 공유했습니다.

SWR 예시

[SWR] SW should provide a maximum of 100 favorites.

SWR Type: Functional · Verification Method: SRT

항목	내용
Description	즐거찾기 최대 100개 제공
Trigger	새 즐겨찾기 추가 시
Flow 1	POI를 검색한다.
Flow 2	POI 검색 결과 목록에서 POI를 선택한다.
Flow 3	POI 스포트라이트 모드에서 즐겨찾기를 저장한다.
Flow 4	즐거찾기가 100개까지 등록될 수 있는 걸 확인한다.
Flow 5	100개 초과 등록 시, 새 즐겨찾기로 덮어쓸지 묻는 팝업이 표시된 것을 확인한다.
Flow 6~7	덮어쓰는 POI를 리스트에서 선택 → 대체한 즐겨찾기가 삭제되고 “새 즐겨찾기 추가” 바로 위에 새 항목이 표시됨

본 예시는 실제 프로젝트의 구조와 유사하게 재구성한 것이며, 모든 내용은 기밀 보호를 위해 가공된 가상 데이터입니다.

위 SWR로 도출한 TC

#	Action	Precondition	Expected Result
1	POI 검색 후 검색 결과 목록에서 POI 선택	—	POI 스포트라이트 화면 표시됨
2	POI 스포트라이트 모드에서 즐겨찾기 저장	—	최대 즐겨찾기 저장 개수가 100개까지 등록됨
3	POI 스포트라이트 모드에서 즐겨찾기 저장	즐거찾기 100개 초과 등록	새 즐겨찾기로 덮어쓸지 묻는 팝업 표시됨
4	덮어쓰는 POI 선택 후 즐겨찾기 목록 확인	—	선택한 즐겨찾기는 삭제되고 “새 즐겨찾기 추가” 바로 위에 새 항목 표시됨

요구사항 문서의 **Basic Flow** 각 단계를 검증 가능한 단위로 쪼개고, 경계(100개 초과) 동작까지 별도 TC로 분리했습니다. 모든 요구사항이 **누락 없이 TC로 커버되도록 추적성을 확보**하는 것을 원칙으로 삼았습니다.

Test Case 기법을 아는 것과 적용하는 것은 다르다

기법 정의보다 실제로 무엇을 만들어 관리했는지를 남깁니다. 탐색적 · 상태 전이 · 시나리오 기반 등도 함께 쓰되, 아래 넷을 기본 축으로 삼았습니다.

기법	적용한 방식	남긴 것
요구사항 기반	요구사항 문서를 항목 단위로 쪼개 빠진 기능이 없는지 대조. 누락 없이 커버되는지를 최우선으로 봄	Jira에 등록 후 요구사항 ID와 매핑 → 추적성 확보
동등 클래스 분할	입력값 범위·조건을 분석해 유효/무효 클래스 로 나누고 대표값 선정. 같은 클래스가 동일하게 처리되는지 확인	필드별 클래스 정의표 — 최소 TC로 최대 결함
경계값 분석	각 필드의 최솟값·최댓값과 직전/직후 를 도출. off-by-one 을 놓치지 않는 것이 목적	min · min-1 · max · max+1 조합
오류 추정	과거 결함 이력 · 복잡한 로직 · 사용자 오용 시나리오를 근거로 의심 영역을 좁힘. 감이 아니라 이력에서 출발	정형 기법으로는 안 잡히는 예외·엣지 케이스

기법을 나눠 관리한 이유 — 어떤 기법으로 만든 TC인지가 남아 있어야 **커버리지의 성격**을 말할 수 있습니다. “TC 300개”보다 “요구사항 전 항목 대조 + 경계값 조합 + 과거 결함 기반 의심 영역”이 훨씬 정확한 진술이고, 회귀 대상을 고를 때도 이 축이 기준이 됩니다.

Defect Ticket 결함티켓 작성 예시

항목	내용
제목	“테스트 벤치 #14”에서 경로 라인 위에 작은 화살표 표시됨
상태 / 우선순위	Closed / Major
보고자 / 담당자	김상욱(와이드넷) / 티맵모빌리티
환경	단말 ID: [가상 단말 ID] · Software Version: v0.25.0.1 · 네트워크: Internet Available 발생 위치: 테스트 벤치 #14 (다른 벤치에서는 미재현)
Precondition	현 위치 서울역
Procedure	1) 목적지 설정 (Test 시에는 부산역으로 설정) 2) 경로 계산 후 경로 라인 확인
Actual Result	주행 중 경로선(Route Line) 위에 작은 화살표 아이콘들이 표시됨 시각적으로 혼란을 유발하며, 디자인 가이드라인과 불일치함
Expected Result	경로선 위에 불필요한 화살표가 표시되지 않아야 함
관련 이슈	과거 동일 UI 이슈가 처리되어 종료된 바 있음. 그러나 일부 특정 장치에서 동일 증상 재현됨 → 회귀 가능성 있음
비고	본 결함은 특정 테스트 벤치에서만 발생하며, 단말 환경 의존적 증상일 수 있음. 팀 내 공유 후 회귀 여부 및 환경 차이 분석 예정

이 티켓이 담고 있는 것 — 단순 현상 보고가 아니라 재현 조건의 경계(어떤 벤치에서만 발생하는가), 과거 이슈와의 연결(회귀 가능성), 다음 액션(환경 차이 분석)까지 명시했습니다.

Xray · Confluence

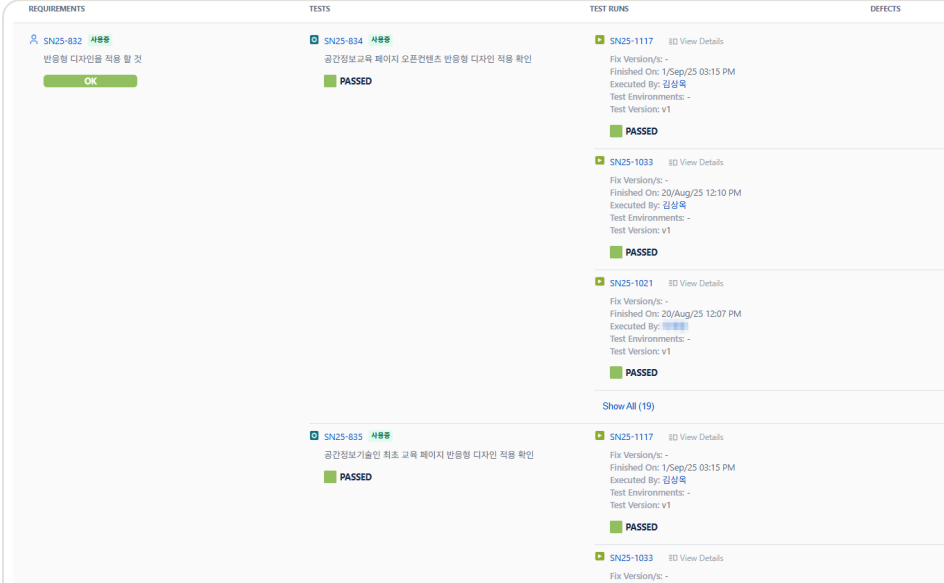
Xray — Test Management for Jira

TC 작성부터 실행·결과 추적까지 **전체 테스트 라이프사이클을 통한 관리**합니다. 요구사항과 TC의 연결로 추적성을 확보하고, 결함 관리 연동으로 효율적 품질 관리가 가능합니다.

| 구성 요소

- **Test Set** — 컴포넌트(기능)별 TC 관리. Xray Importer로 TC 일괄 업로드
- **Test Plan** — 테스트 스케줄 관리 및 수행 관리
- **Test Execution** — Test Run 수행 결과 기록

| 추적성 관리



Requirement Traceability — 요구사항 하나에서 연결된 TC와 각 버전의 실행 결과까지 한 줄로 따라간다.

Confluence 산출물

회사 보안 정책에 따라 외부 공유가 제한되며, 본 포트폴리오는 문서 요약 및 활용 목적에 한하여 작성되었습니다.

| Test Review 리포트

- 요구사항 명세서 리뷰 리포트
- 성능 KPI 측정 리포트 (비기능 테스트)
- Test Plan
- Test Report (Sprint Test, 기타)

| 회의록

- 주간 회의 · 고객사 미팅 · 스프린트 회의 등

| 자동화 테스트 문서

- 자동화 스크립트 작성 기록 · 배포 게시글
- 배포한 프로그램 가이드 등

| Test Guide 문서

- 로그 추출 방법 가이드 · 명령어 가이드 · 테스트 방법 가이드

Defect Traceability Report

Filters

Filter Defects

Projects: 2025 SPACEIN

Connected defects

Show Test Runs

QUICK FILTERS (무인 순차)

Highest

High

Medium

Low

Lowest

DEFECTS	TEST RUNS	TESTS	REQUIREMENTS
SN25-1120 연락통 Assignee: 김민준 Fix Version/s: - Safari Browser에서 화면가림 시 아이디 입력폼에 틀린값 ...	SN25-1116 ID View Details Fix Version/s: - Finished On: 1/Sep/25 11:34 AM Executed By: 김성욱 Test Environments: - Test Version: v1 FAILED	SN25-775 사용중 Assignee: - Fix Version/s: - 화면가림에서 작성하는 입력폼에 값이 잘못된 상태로 화면...	SN25-666 사용중 Assignee: - Fix Version/s: - 입력폼 값 오류시 제어 흐름 방법
SN25-1112 연락 Assignee: 김민준 Fix Version/s: 0.04.1 Safari Browser에서 수강취소 시 에러메시지 진입	SN25-1115 ID View Details Fix Version/s: - Finished On: 1/Sep/25 12:17 PM Executed By: 김성욱 Test Environments: - Test Version: v1 FAILED	SN25-1118 사용중 Assignee: 김성욱 Fix Version/s: - 나의 검색결과 - 강의 상세 페이지에서 수강취소가 되어야 한다	
SN25-1064 연락 Assignee: 김민준 Fix Version/s: 0.03.5 iOS 검색창에서 검색박스에 띄워보장 없음	SN25-1011 ID View Details Fix Version/s: 0.03.4 Finished On: 20/Aug/25 06:01 PM Executed By: 김성욱 Test Environments: - Test Version: v1 PASSED	SN25-1036 사용중 Assignee: 김성욱 Fix Version/s: - 문맥내역 페이지 반응형 디자인 적용 확인	SN25-832 사용중 Assignee: 김성욱 Fix Version/s: - 반응형 디자인을 적용 할 것
SN25-1059 연락 Assignee: 김민준	SN25-1021 ID View Details Fix Version/s: -	SN25-758 사용중 Assignee: -	SN25-662 사용중 Assignee: -

Defect Traceability — 결함에서 역방향으로 실행 이력 · TC · 요구사항까지 되짚는다. (담당자명 비식별 처리)

- **Requirement Traceability** — 모든 요구사항이 TC로 연결되어 충족 여부 판단 가능
- **Defect Traceability** — 모든 Defect 티켓이 TC와 연결되어 요구사항·스토리까지 추적 가능
- Xray Report로 리포트 생성

화면에서 막힌다고 서버에서도 막히는 것은 아니다

프론트엔드 유효성 검증은 **사용자에게 친절한 장치일 뿐 품질 보증이 아닙니다**. 그래서 UI를 거치지 않고 **REST API를 직접 호출해** 같은 규칙이 서버에서도 지켜지는지 확인했습니다.

검증 절차 — 캡처에서 증거까지

1. 요청 캡처

Chrome DevTools로 실제 패킷 추출



2. 구조·인증 분석

파라미터 구조 · Cookie · CSRF Token



3. 변조 요청

Postman · cURL로 값·형식·순서 조작



4. 결과 검증

응답 · DB 저장 · 관리자 화면 반영까지

무엇을 보는가 — 5개 축

축	확인하는 것
입력 검증	UI가 막는 값(형식·길이·범위)을 서버도 막는가 — 우회 시 그대로 통과하는가
인증 · 인가	Cookie-CSRF Token 없이/타인 것으로 호출하면 어떻게 되는가
데이터 무결성	규칙 위반 값이 DB에 실제로 저장되는가 , 이후 화면에 어떻게 나오는가
오류 응답	실패 시 상태 코드·메시지가 일관된가, 내부 정보를 흘리지 않는가
보안 취약점	스크립트·특수문자 주입이 저장·렌더링 경로까지 이어지는가

왜 이렇게

UI 자동화는 **“막히는 것”만 확인**합니다. 화면에서 막혔다는 사실은 서버가 막았다는 증거가 아니라 **화면이 요청을 보내지 않았다는 증거일** 뿐입니다. 같은 시나리오를 API로 한 번 더 두드려야 비로소 **“검증했다”고 말할 수** 있다고 봤습니다.

사용한 도구

Postman

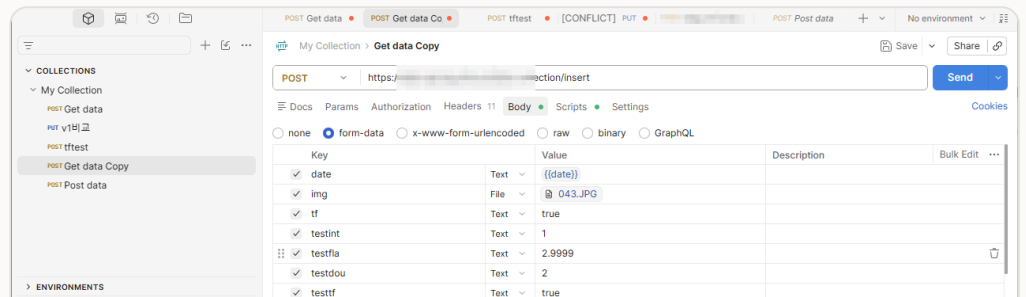
cURL

Chrome DevTools

Burp Suite

Lighthouse

화면을 거치지 않기 때문에 가능한 것



API 계층에서 타입·정밀도 경계값을 직접 넣어 서버의 수용 범위를 확인한다. (호스트 비식별)

- 타입·정밀도 경계를 그대로 통과시켜 본다 — 정수 필드에 실수를, 불리언에 문자열을 넣었을 때 **서버가 무엇을 저장하는지**가 서버 측 검증 부재를 드러내는 지점이었습니

- **도달하기 어려운 상태를 만들어 놓고 시작한다** — 배포 상태 같은 값을 직접 PUT으로 세팅하면 UI로 몇 단계 거쳐야 하는 전제 조건을 한 번에 만들 수 있습니다
- **거부돼야 할 요청을 만들어 보낸다** — 인증 헤더를 빼거나 바꿔서, 통과하는 경우가 아니라 **막혀야 하는 경우**를 확인합니다

▮ 적용한 곳

- **공간정보진흥원 LMS** — 회원가입·회원정보 등 주요 API 전반 검증, 웹 접근성(KRDS)·보안 취약점 점검과 병행
- **뉴퍼마켓(LLM ERP 매칭)** — HTTP 통신 보안 취약점 조사 후 개선 사항 도출
- **DAISY** — 장애 주입 검증에서 요청 차단·응답 조작으로 실패 경로 재현 Ch.02 MLOps E2E

발견 사례 — 프론트만 막고 있던 유효성 검증

UI에서는 정상적으로 거부되던 값들이, **API**를 직접 호출하면 그대로 저장됐습니다.

진행 어떻게 확인했나

- DevTools로 회원가입 · 회원정보 수정 요청 패킷 추출
- 파라미터 구조와 인증 방식(Cookie · CSRF Token) 분석
- Postman·cURL로 이메일 · 비밀번호 · 연락처 · 성별 필드에 형식 위반 값과 `<script>` 태그를 삽입해 전송
- 응답만 보지 않고 **DB 저장 여부와 관리자 페이지 반영까지** 추적

발견 무엇이 문제였나

- **서버 측 입력값 검증 부재** — 프론트엔드 검증만 존재했고 API 레벨 방어가 없었음
- 형식·규칙을 위반한 데이터가 그대로 **DB에 저장됨** → 데이터 무결성 훼손
- 주입한 스크립트가 저장을 거쳐 **관리자 페이지에서 렌더링되는** 경로 확인 → XSS 취약점

사용자 화면에서만 막히는 검증은, **요청을 직접 보낼 수 있는 사람 앞에서는 없는 것과 같습니다.**

보고 어떻게 넘겼나

- **재현 절차를 그대로 옮길 수 있게** 정리 — 요청 원문·변조 지점·기대 응답
- **스크린샷과 패킷 로그를 증거로 확보** — “된다/안 된다” 논쟁을 없애기 위해
- UI 결함이 아니라 **서버 검증 로직 부재**라는 원인으로 규정해 티켓 작성
- 개선 방향(서버 측 검증 추가·저장 시 이스케이프)까지 제안해 수정 주도

남은 원칙

그 뒤로 품이 있는 기능은 **UI 통과와 API 통과를 별도의 검증 항목으로** 나눠 씁니다. 같은 규칙을 두 계층에서 각각 확인하지 않으면 통과했다고 보지 않습니다.

이 건이 보여주는 것 — 화면 기준 테스트만 했다면 끝까지 발견되지 않았을 결함입니다. QA가 UI 레벨에 머무르지 않고 **API 레벨까지 내려가 보안 품질을 강화한** 사례이고, 지금도 API 검증을 기능 검증의 선택 항목이 아니라 기본 축으로 두는 이유입니다.

REST API 검증

Postman · cURL

파라미터 변조

서버 측 검증 부재

XSS

데이터 무결성

증거 기반 리포트

업무 자동화 도구 로그 · 데이터 수집

반복 업무를 직접 개발한 스크립트로 줄였습니다. 일부 저장소는 보안 이슈로 **면접 시 제한적 공개**합니다.

업무용 종합추출도구

Python · paramiko · Tkinter

원격 서버에 SSH로 접속하여 로그 파일, 스크린샷, CoreDump, Binary 등을 자동으로 다운로드 및 제어하는 GUI 도구입니다. IP를 선택하고 작업을 클릭하면 진행 상황이 표시되며 결과가 자동 저장됩니다.

| 구현 기능

- SSH 원격 접속 후 로그 및 스크린샷 등 파일 수집
- DLT · CoreDump · Position 로그 다운로드 및 자동 압축
- Navigation 서비스 중단 및 재시작 자동화
- 바이너리 업로드 · 압축 해제 · 권한 설정 자동화
- JSON 설정 파일 확인 기능
- **GUI 기반으로 비전문가도 사용 가능**

KPI Calculator

Python · re · threading · Tkinter

DLT 로그 파일에서 네비게이션 시스템의 KPI LOG를 자동 분석하고, GUI 버튼 클릭만으로 각 항목의 시간 차를 시각적으로 확인할 수 있는 도구입니다.

| 구현 기능

- `.dlt` 로그 파일에서 특정 키워드를 정규식으로 파싱
- 이벤트 간 시간 차 계산 (NAV_START · NAV_SEARCH · NAV_ROUTE · NAV_TRAFFIC_INFO)
- `stdout` 출력을 ScrolledText 위젯에 실시간 표시
- Thread를 활용해 **GUI 멈춤 없이** 로그 분석 진행

연계 성과 — 비기능 KPI 측정 스크립트 개발로
수작업 대비 검증 리소스 **30% 절감**

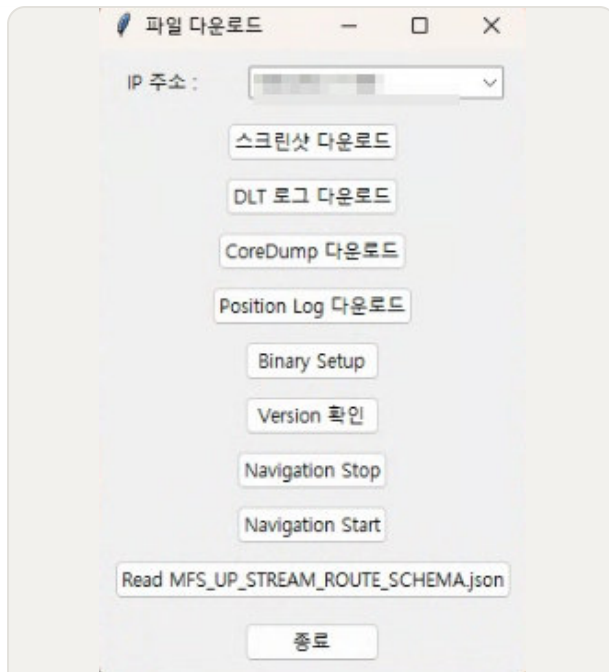
Positioning Viewer

Python · pandas · folium · Tkinter

GNSS 로그 파일에서 위도/경도 좌표를 추출해 CSV로 저장하고, Folium으로 지도 위에 경로를 시각화하는 도구입니다.

| 구현 기능

- GNSS 로그(`*.log`)에서 `[GNSS]` 키워드 라인 필터링
- 위도/경도 추출 및 **유효성 검사**를 통해 좌표 리스트 생성
- 추출된 좌표 데이터를 CSV로 저장
- 지도 상에 경로를 폴리라인(PolyLine)으로 시각화
- 생성된 HTML 파일을 기본 브라우저에서 자동 실행
- Tkinter GUI로 복수 파일 선택 가능



IP를 고르고 버튼 하나로 로그·스크린샷·CoreDump를 받아온다. 서비스 중단/재시작과 바이너리 배포까지 같은 창에서 처리. (장비 IP 비식별)

업무 자동화 도구 번역 · 알림

Clipboard Translate

Python · Tesseract OCR · Google Translate

클립보드에 복사된 이미지나 텍스트를 **0.5초 간격으로 실시간 감지**하여 OCR과 번역을 자동 수행하는 데스크톱 자동화 도구입니다. JLPT N2 일본어/영문 문서 작업을 가속했습니다.

| 핵심 기능

- **스마트 텍스트 병합** — OCR로 인한 불필요한 줄바꿈을 문장 종결 부호 기준으로 지능적 제거
- **명사형 종결 변환** — 기술 문서 스타일로 30개 이상 규칙 적용
"제거해야 합니다" → "제거 요망"
- **다중 모드** — F7(OCR 전용) / F8(OCR+번역) / F9(일시정지) 상호 배타적 전환
- **중복 처리 방지** — 이미지 픽셀 데이터 비교 및 텍스트 캐싱으로 **무한 루프 차단**
- **자동 경로 탐지** — Tesseract 설치 위치를 4단계로 자동 탐색

성과 — 기존 5단계 작업(캡처 → 저장 → OCR 실행 → 복사 → 번역기 입력)을 **1단계(캡처)**로 단축, 평균 2~3초 내 완료

공수표 알리미

Google Apps Script × Slack Workflow

"매달 잊지 말고 공유해야 하는 링크"를 **사람의 기억에서 시스템으로** 옮긴 사내 자동화입니다.

| 문제

매달 새로 생성되는 공수표 URL을 담당자가 기억해서 팀에 수동 공유하는 구조. 잊으면 누락, 늦으면 팀 전체의 입력 지연 — 전형적인 **"사람이 크론잡 역할을 하는"** 업무였습니다.

| 구성

- **Google Apps Script** — 월별 URL 시트를 조회해 매주 금요일 그 달의 URL 자동 발송
- **Slack Workflow 연동** — Slack 앱 생성부터 Incoming Webhook 셋업까지 직접 구축
- **이중 안전장치** — 해당 월 링크가 없으면 즉시 경고, 매월 25일에는 다음 달 준비 여부 사전 점검

QA 산출물은 아니지만 일하는 방식은 같습니다 — **반복되는 것은 자동화하고, 실패 케이스(링크 누락)까지 설계에 포함**합니다. 알림이 실패하는 경우를 알림으로 잡는 구조까지 넣어야 "자동화했다"고 말할 수 있다고 생각합니다.

기술 스택

테스트 관리 · 협업

Jira

Jira Xray

Jira Automation

ClickUp

Confluence

Slack

Notion

테스트 자동화 · 스크립팅

Playwright

Selenium

Postman

Python

Node.js

Shell Script

Jenkins

GitHub Actions

AI · LLM 활용

Claude Agent

MCP (ClickUp · Google Sheets)

생성형 AI 업무 자동화

LLM 응답 정확도 검증

자동차 HILs · 진단

Vector CANoe

CANalyzer

Monaco

ETFW

DCM 연동

웹 보안 · 품질 진단

Burp Suite

Lighthouse

KRDS 웹 접근성

로그 분석 · 모니터링

DLT Viewer

Android ADB

Python/Shell 커스텀 로그 필터

검증 환경 · 플랫폼

Linux 기반 IVI

HILs 장비

실차 테스트

Web / Mobile

AI/ML Vision Inspection

LLM 기반 시스템

서버리스 · 백엔드 연동

AWS Lambda

Slack API

ClickUp API

Google Sheets API

Google Apps Script

기타

Git

GitHub

Figma

TypeScript

토이 프로젝트

업무 산출물은 아니지만, 만드는 습관이 시작된 곳

회사 업무와 무관하게 **궁금해서 만들어 본 개인 프로젝트**입니다. 업무 성과로 내세울 것은 아니지만, 앞서 본 실무 도구들이 어디서 왔는지를 보여주는 자리라 함께 남겨둡니다.

NASDAQ AUTO — Multi-Agent AI 시스템

TypeScript · Node.js · 개인 프로젝트

3개의 서로 다른 AI 모델(GPT · Gemini · Claude)이 각자의 관점에서 분석하고, **Manager Agent가 세 보고서를 통합해 최종 결론을 내리는** Multi-Agent 협력 구조를 직접 설계·구현했습니다.

구조

Agent	역할
Agent_GPT	체계적·정량적 분석, 복합 데이터 해석
Agent_Gemini	빠른 분석, 실시간 트렌드·모멘텀
Agent_Claude	심층적·보수적 분석, 리스크 관리
Manager_Agent	3개 보고서 종합 → 최종 판단

만들면서 얻은 것

- 완전 자동화 파이프라인(22개 모듈) — 스케줄러 → 데이터 수집 → 3 Agent 병렬 실행 → 통합 → 리포트 발송
- 데이터 무결성 설계 — 입력 검증, 초과 처리 방지, 중요 작업 전 자동 백업 및 복구
- 판단 근거의 추적성 — 각 Agent의 분석 과정과 근거를 전부 기록해 사후 검증 가능하게

QA 관점에서 남은 것 — 서로 다른 모델의 관점을 교차 검증해 편향을 줄이는 구조와, 자동 판단에는 반드시 근거 기록이 따라와야 한다는 원칙. 이 감각이 이후 Regression TC Creator의 안전장치 설계로 이어졌습니다.

github.com/kx2471/NASDAQ_AUTO ↗

OCR 활용 매크로

Python · Pillow · pytesseract · Google Tesseract · 개인 프로젝트

OCR 기술과 Tesseract에 관심이 생겨 **직접 만들어 본 개인 도구**입니다. 지정한 화면 영역을 일정 간격으로 캡처하고, 그 영역에서 텍스트를 자동 추출합니다.

구현 기능

- 특정 화면 영역 주기적 캡처 (`bbox` 좌표 설정 가능)
- Tesseract OCR(`--psm 11`)로 이미지에서 텍스트 추출
- 추출 텍스트를 UTF-8 파일로 저장
- 캡처 이미지를 로컬에 보존하여 **디버깅 가능**
- 설정된 간격(예: 3초)마다 자동 반복 실행

여기서 이어진 것

이때 익힌 OCR 파이프라인이 그대로 실무 도구로 넘어갔습니다 — **Clipboard Translate**(Ch.05)의 이미지 텍스트 추출부가 이 실험에서 출발했고, OCR 기반 비정형 UI 테스트 자동화라는 아이디어도 여기서 얻었습니다.

이 두 프로젝트를 남겨두는 이유 — 업무 성과가 아니라 “**궁금하면 만들어 본다**”는 습관 자체가 실무 도구의 출처이기 때문입니다. OCR 매크로 → 클립보드 번역기, Multi-Agent 실험 → QA 워크플로우용 Agent로 이어졌습니다.

NASDAQ AUTO는 개인 학습 목적의 프로젝트이며 투자 참고용으로만 제작되었습니다. 투자자문이 아닙니다.

THANK YOU

품질은 작은 습관에서 시작된다고 믿습니다

꼼꼼한 준비와 일정 관리를 업무 습관으로 유지하며 지연 없이 프로젝트를 완수해왔습니다. 체계적인 품질 관리와 끊임없는 개선 활동을 통해 소프트웨어 품질 향상에 **실질적으로 기여**하고자 합니다.



010-4482-2033



kx2471@gmail.com



github.com/kx2471

본 포트폴리오의 문서와 도식은 실제 업무 경험을 바탕으로 재구성·가공되었으며, 회사 기밀 보호를 위해 고객사·버전·수치 등 모든 민감 정보는 익명화하거나 예시 패턴으로 대체되었습니다. 일부 저장소는 보안 정책에 따라 면접 시 제한적으로 공개합니다.

김상옥 · Kim Sang-Ok — SW QA Engineer